

大语言模型应用

第 5 章 检索增强生成与向量数据库

讲 义

伍孝金

2026 年 3 月 9 日

目录

第 5 章 检索增强生成与向量数据库.....	4
5.1 什么是检索增强生成.....	4
5.1.1 为什么需要 RAG：从“闭卷”到“开卷”的范式转变.....	4
5.1.2 RAG 的定义与核心架构.....	5
5.1.3 RAG 的三大范式.....	7
5.1.3 构建基础 RAG 系统.....	8
5.2 向量数据库的核心概念与技术.....	14
5.2.1 从传统数据库到向量数据库.....	14
5.2.2 为什么 RAG 需要向量数据库.....	15
5.2.2 向量数据库的基本原理.....	16
5.2.3 主流向量数据库生态概览.....	16
5.3 向量数据库 Chroma 的使用.....	18
5.3.1 ChromaDB 的安装与设置.....	18
5.3.2 核心概念：集合与文档.....	19
5.3.3 核心操作：数据的增删改查 (CRUD).....	19
5.3.4 查询与过滤.....	21
5.3.5 RAG 使用 Chroma 向量数据库.....	21
5.3.6 Chroma 支持的嵌入模型.....	25
5.4 高级 RAG（Advanced RAG）.....	30
5.4.1 问题驱动：朴素 RAG 的局限.....	31
5.4.2 方法框架：高级 RAG 整体架构.....	31
5.4.3 高级 RAG 的索引阶段优化.....	32
5.4.4 高级 RAG 的检索阶段优化.....	33
5.4.5 高级 RAG 的生成阶段优化.....	33
5.4.6 综合实现：高级 RAG 系统示例.....	34
5.5 RAG 系统的评估简介.....	37
5.5.1 RAG 系统的评估标准.....	37
5.5.2 RAG 系统的评估步骤.....	38
5.6 案例实战：基于 RAG 的智能知识管理与问答系统.....	40
5.6.1 智能知识管理与问答系统的功能需求.....	40
5.6.2 智能知识管理与问答平台的总体设计.....	42
5.6.3 知识库模块的实现.....	44
5.6.4 检索器模块的实现.....	46
5.6.5 生成器模块的实现.....	47

5.6.6 UI 界面	48
5.6.7 程序主入口和运行	51
5.6.8 运行示例.....	52
本章小结.....	54
阅读文献和资料.....	54

第 5 章 检索增强生成与向量数据库

随着大语言模型的迅猛发展，其在自然语言理解、生成和交互等任务中展现出惊人的能力。然而，大语言模型并非万能。它们固有的“知识截止日期”问题、在特定领域知识上的匮乏，以及“幻觉”现象，都限制了其在许多关键应用场景中的可靠性。为了克服这些挑战，检索增强生成（Retrieval-Augmented Generation, RAG）技术应运而生，它巧妙地将大语言模型与外部知识库相结合，极大地提升了模型回答的准确性、时效性和可解释性。

向量数据库是专门用于存储和检索高维向量表示（即嵌入），支持语义级别的相似性搜索，是连接非结构化文本与 LLM 的桥梁实现，是实现 RAG 系统的关键技术之一。近年来，Chroma、Pinecone、Weaviate、Milvus 等向量数据库迅速发展，为 RAG 应用提供了强大支撑。

本章将系统讲解 RAG 的基本原理、向量数据库的核心技术、主流工具 Chroma 的使用方法，并深入探讨生产环境中的优化策略、性能评估指标，最后通过一个完整的智能问答系统应用案例的开发，掌握 RAG 系统开发的流程，为后续 RAG 技术打下坚实的基础。

【本章学习目标】

- 理解 RAG 技术的核心思想与价值：能分析 LLM 局限性，阐释 RAG “检索-生成”机制，实现基础 RAG 系统的核心功能。
- 掌握向量数据库的关键概念：理解向量嵌入、相似度度量、近似最近邻搜索（ANN）等核心原理；了解主流的向量数据库。
- 精通 Chroma 向量数据库的使用：能够熟练运用 Chroma 进行数据的增删改查、元数据过滤和自定义嵌入函数集成；实现在基础 RAG 系统运用 Chroma 存储向量数据。
- 具备设计生产级 RAG 系统的能力：了解并能够应用数据分块、混合检索、查询转换、重排等高级优化策略。
- 初步掌握 RAG 系统的评估方法：了解 RAG 系统评估的标准、指标体系和步骤，能够运用 RAGAs 等框架，从检索质量、生成忠实度等多个维度科学地评估 RAG 系统。
- 具备开发基础 RAG 系统的能力：能够独立完成一个端到端的智能问答系统的设计、开发、部署与评估。

5.1 什么是检索增强生成

随着大语言模型在实际应用中的深入，人们发现通用大模型在处理私有知识、动态信息和高可信度要求场景时存在明显局限。检索增强生成（Retrieval-Augmented Generation, RAG）通过“检索和生成”的协同模式，有效解决了这些难题。本节将从产生背景、核心定义、工作流程及技术演进四个维度，系统解析 RAG 技术。

5.1.1 为什么需要 RAG：从“闭卷”到“开卷”的范式转变

1. 大模型落地的三大核心痛点

尽管大模型在通用任务上表现出色，但在企业级核心场景中往往难以直接落地，主要面临以下挑战：

- 知识滞后（Knowledge Latency）：模型训练数据存在截止时间（Cut-off Date），无法回答训练之后的最新政策、新闻或行业动态。
- 幻觉频发（Hallucination）：当缺乏确切事实依据时，模型可能生成看似合理但实际错误的内容，这在法律、医疗等严谨场景中是不可接受的。
- 知识脱节（Data Silos）：出于数据安全与隐私保护考虑，企业无法将敏感文档（如内部手册、涉密数据）上传至公有云模型进行训练或微调。

2. 根本矛盾与解决方案

深入分析可知，上述问题的根源在于：大模型静态固化的参数化知识，与现实业务动态变化的需求之间存在不可调和的矛盾。模型训练完成后，其知识体系即被锁定，无法自主获取新信息。

为了解决这一矛盾，2020 年，Facebook AI（现 Meta AI）研究团队在开创性论文《Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks》中正式提出了检索增强生成（RAG）架构。

- 核心理念：将大模型从“闭卷考试”转变为“开卷作答”。
- 运作模式：在生成答案前，先通过检索模块从外部知识库（如企业私有文档库、实时数据库）中精准抓取相关信息片段，再将这些“参考资料”与用户问题一同提交给大模型。
- 核心价值：
 - ✓ 根治幻觉与滞后：答案源于外部最新、最准确的数据，从源头杜绝捏造。
 - ✓ 保障数据安全：敏感数据无需离开本地环境，仅通过检索接口交互。
 - ✓ 低成本动态更新：无需昂贵的模型微调，仅更新外部知识库即可实现知识迭代。

5.1.2 RAG 的定义与核心架构

1. 标准定义

RAG 是一种融合外部知识检索与大语言模型生成能力的人工智能系统架构范式。根据 Meta AI 的定义，它是“一种通过从大型外部语料库中检索相关文档，并将这些文档作为条件上下文输入给序列到序列生成模型，以提升其在知识密集型任务中表现的框架”。

简言之，RAG 不让大模型仅依赖内部记忆作答，而是为其配备一个可实时更新、可审计的“外部图书馆”。

2. 核心工作流程

一个标准的 RAG 系统包含五个环环相扣的关键步骤，如图 5.1 所示。

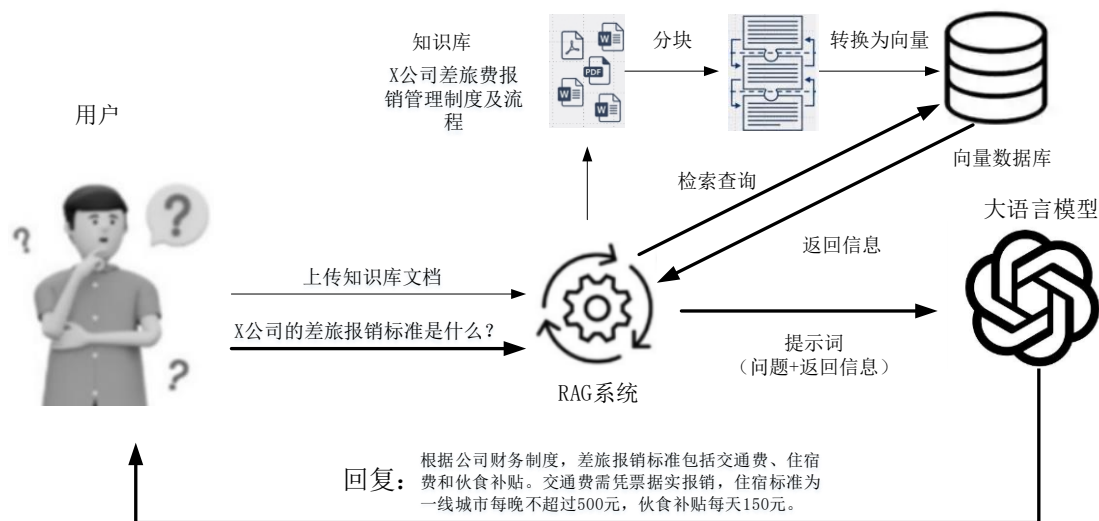


图 5.1 RAG 系统工作流程示意图

（1）文档处理

用户（一般是 RAG 系统管理员）上传知识库文档，如 X 公司差旅费报销管理制度及流程等 pdf 文件，RAG 系统将原始非结构化或半结构化数据（如 PDF、Word、网页、FAQ、数据库记录等）解析为纯文本格式。此阶段需进行格式清洗、元数据提取（如标题、作者、时间戳）及敏感信息脱敏。

（2）文本分块

将长文档按语义边界（如段落、章节）或固定长度（通常 300 – 800 字符）切分为若干分块（Chunking），并保留适度重叠（如 50 字符），以避免关键信息被割裂。如将 X 公司差旅费报销管理制度及流程等 pdf 文件进行分块。

（3）向量化与索引构建

使用预训练的文本嵌入模型（如 Sentence-BERT、BGE、text-embedding-ada-002）将每个文本块映射为高维稠密向量（例如 768 维或 1536 维）；然后，将所有文本块及其对应的向量、元数据一并存入向量数据库（如 Chroma、Milvus、Qdrant）。数据库内部会构建高效的近似最近邻（ANN）索引结构（如 HNSW、IVF-PQ），以支持毫秒级的语义相似度搜索。如将 X 公司差旅费报销管理制度及流程的分块转化为向量，保存在向量数据库中。

（4）相关性检索

当用户提问时，系统将问题转化为查询向量，在向量数据库中执行语义相似度搜索，召回最相关的 Top-K 个文本片段。当员工在聊天界面提问：“公司的差旅报销标准是什么？”，系统首先通过相同的嵌入模型将此问题转换为查询向量。然后，在向量数据库中执行 ANN 搜索，从海量的内部文档中，返回与该问题语义最相似的 Top-K 个文本片段（即“召回”阶段）。

（5）提示构造与答案生成

将原始问题与检索到的 Top-K 文本块按预设模板拼接为结构化提示（Prompt），输入大语言模型。模型基于提供的证据生成准确、可追溯的回答。

例如：

请根据以下公司资料回答员工问题。若资料中未提及，请回答“无法根据现有资料回答”。

资料：

{检索到的报销标准条款 1}

{检索到的报销流程条款 2}

...

员工问题：公司的差旅报销标准是什么？

回答：

最后，将该提示输入大语言模型。大语言模型基于提供的上下文，生成一个简洁、准确的回答，例如：“根据公司财务制度，差旅报销标准包括交通费、住宿费和伙食补贴。交通费需凭票据实报销，住宿标准为一线城市每晚不超过 500 元，伙食补贴每天 150 元。”系统还可以附上原文档的链接供员工查阅。

3. 三大核心组件

在上述流程中，三个组件扮演着至关重要的角色：

- 知识库：系统的“外部记忆体”，存储所有可供检索的事实来源。其优势在于内容可随时更新，确保时效性。
- 检索器：系统的“眼睛”，负责理解用户意图并从海量数据中精准定位相关信息。现代检索器多基于稠密向量技术，能捕捉深层语义而非仅关键词匹配。
- 生成器：通常是大语言模型本身。在 RAG 中，它的角色从“知识记忆者”转变为“信息整合者”，负责基于检索到的证据进行推理和作答。

5.1.3 RAG 的三大范式

随着应用的深入，RAG 架构也在不断进化，主要经历了三个阶段：

- 基础 RAG (Naive RAG)：遵循标准的“索引→检索→生成”流程。架构简单，易于实现，但在处理复杂查询时可能存在检索精度不足的问题。（注：本章 5.3 节将手把手构建一个 Naive RAG 系统）
- 高级 RAG (Advanced RAG)：在基础之上引入优化策略，如检索前的查询重写、检索中的混合搜索（关键词+向量）、检索后的重排序（Rerank）等，显著提升准确率。（注：详见 5.4 节）

- 模块化 RAG (Modular RAG): 最新的发展方向, 采用可插拔的模块化设计, 允许动态组合路由、记忆、多跳推理等模块, 以应对极度复杂的应用场景。(注: 将在第 6 章进行介绍)。

5.1.3 构建基础 RAG 系统

理解了 RAG 的工作流程和三大核心组件后, 下面将构建一个基础的 RAG 系统。这个基础的 RAG 系统从构建一个小型中文知识库开始, 依次实现检索器的向量化索引、基于语义的文档检索, 以及调用大语言模型生成答案的完整流程, 最终完成一个端到端的基础 RAG 系统搭建(“端到端”指的是: 用户只需输入一个问题, 系统就能自动完成检索和生成, 直接输出最终答案, 整个过程一气呵成)。

项目 simple-rag 的结构如下:

```
simple-rag/
├── main.py          # 主程序, 集成 RAG 流程
├── knowledge.py     # 知识库模块, 存储文档数据
├── retriever.py     # 检索模块, 语义检索功能
├── generator.py     # 生成模块, 调用 API 生成答案
├── .env             # 环境变量配置文件
└── README.md       # 项目说明文档
```

项目结构说明:

- main.py : 主程序, 集成完整 RAG 流程, 包括加载知识库、检索相关文档、生成答案
- knowledge.py : 知识库模块, 存储中国传统文化相关文档(春节、汉字、京剧、茶文化、中医药)
- retriever.py : 语义检索模块, 使用 SentenceTransformer 库的 all-MiniLM-L6-v2 模型, 通过 encode_documents() 函数将文档转换为嵌入向量
- generator.py : 生成模块, 调用 DeepSeek API 生成答案
- .env : 环境变量配置文件, 存储 API 密钥
- README.md : 项目说明文档

源码:

https://gitee.com/wu-solo_admin_admin/llm-course/tree/master/chapter05/simple-rag

下面重点介绍知识库、检索器、生成器和主程序模块的实现过程。

1. 构建知识库

程序清单 5.1 用于创建一个关于中国文化的简单知识库, 作为后续检索与生成任务的数据源。

程序清单 5.1 knowledge.py。

```

1.  # knowledge.py
2.  # 中国传统文化知识库
3.
4.  documents = [
5.      "春节是中国最重要的传统节日，日期为农历正月初一，有贴春联、放鞭炮、吃年夜饭等习俗。",
6.      "汉字是世界上唯一仍在广泛使用的表意文字系统，已有 3000 多年历史。",
7.      "京剧是中国国粹，形成于北京，已有 200 多年历史，以唱、念、做、打为表演特色。",
8.      "中国茶文化源远流长，茶叶分类包括绿茶、红茶、乌龙茶、黑茶、白茶、黄茶六大类。",
9.      "中医药是中国传统医学，包括中药、针灸、推拿等治疗方法，强调阴阳平衡和整体观念。"
10. ]

```

程序清单 5.1 用一个 documents 列表保存了这个最简单的关于中国文化的几条数据。

2. 实现检索器

检索器是 RAG 系统的核心部分，负责从知识库中查找与用户问题最相关的信息。这个过程本质是语义检索，它不是简单的关键词匹配，而是理解查询和文档的含义，然后找到语义上最相似的文档。这里的最相似一般是指计算余弦相似度。

程序清单 5.2 实现了 RAG 系统中检索器组件的功能。

程序清单 5.2 retriever.py。

```

1.  # retriever.py
2.  # 语义检索模块
3.
4.  from sentence_transformers import SentenceTransformer, util
5.  import numpy as np
6.
7.  # 初始化嵌入模型
8.  embedding_model = SentenceTransformer('all-MiniLM-L6-v2')
9.
10.
11. def encode_documents(documents):
12.     """将文档转换为嵌入向量"""
13.     return embedding_model.encode(documents, convert_to_tensor=True)
14.
15.
16. def retrieve(query, documents, doc_embeddings, top_k=2):
17.     """检索相关文档"""
18.     # 编码查询

```

```
19.     query_embedding = embedding_model.encode(query, convert_to_tensor=True)
20.
21.     # 计算余弦相似度
22.     hits = util.pytorch_cos_sim(query_embedding, doc_embeddings)[0]
23.
24.     # 获取最相关的 top_k 文档索引
25.     top_indices = np.argsort(hits.numpy())[::-1][:top_k]
26.
27.     # 返回相关文档
28.     return [documents[i] for i in top_indices]
```

程序清单 5.2 中的检索器主要实现了三个方面的任务：

- 函数 `encode_documents(documents)` 将文本转换为数值向量：使用预训练的语言模型将文本（查询和文档）转换为高维向量（称为嵌入向量或嵌入表示）。
- 函数 `retrieve()` 中的第 22 行是计算语义相似度：通过比较查询向量和文档向量的相似度，找到语义上最接近的文档。
- 函数 `retrieve()` 中的第 25、28 行时获取和返回最相关文档：根据相似度排序，返回最相关的 K 个文档。

函数 `retrieve()` 是 RAG 检索器的核心函数，负责从知识库中查找与用户问题相关的文档。

下面详细解释每个参数的含义和作用：

（1）`query`（查询）：用户提出的问题或查询语句，比如：“中国茶叶分为哪些种类？”，这是检索的起点，整个检索过程都围绕这个查询展开，在函数内部，`query` 会被转换为嵌入向量，然后与知识库文档的向量进行比较

（2）`documents`（文档）：原始的知识库文档集合，在这个代码中，就是 `documents` 列表，每个字符串代表一个知识条目，这是 RAG 系统的“记忆”或“知识库”，函数的最终目标就是从这些文档中找出与查询最相关的部分。

（3）`doc_embeddings`（文档嵌入向量）：知识库文档对应的向量表示，这是 `documents` 中每个文档的数值化表示，每个文档被转换为一个高维向量（如 384 维），这些向量是通过预训练的嵌入模型生成的，语义相似的文档在向量空间中距离较近。

（4）`top_k`（返回文档数量）：需要返回的最相关文档的数量，默认值 2 表示返回最相关的 2 个文档，可以根据需要调整，比如设置为 1、3、5 等。

3. 实现生成器

生成器是 RAG 系统的最终输出环节，负责根据检索器找到的相关信息，生成自然语言回答。这个组件体现了 RAG 系统的“生成”部分，它将检索到的知识转化为人类可读的回答。

程序清单 5.3 实现了 RAG 系统中的生成器组件。

程序清单 5.3 generator.py。

```
1. # generator.py
2. # 答案生成模块
3.
4. import requests
5. import os
6.
7.
8. def generate_answer(question, context):
9.     """调用 DeepSeek API 生成答案"""
10.    context_str = "\n".join(context)
11.
12.    # 构建提示词
13.    prompt = f"""基于以下信息回答问题：
14.    {context_str}
15.
16.    问题: {question}
17.
18.    回答: """
19.
20.    # API 配置
21.    api_key = os.getenv("DEEPSEEK_API_KEY")
22.    headers = {
23.        'Content-Type': 'application/json',
24.        'Authorization': f'Bearer {api_key}'
25.    }
26.
27.    data = {
28.        "model": "deepseek-chat",
29.        "messages": [
30.            {"role": "system", "content": "你是一位中国传统文化专家。"},
31.            {"role": "user", "content": prompt}
32.        ],
33.        "max_tokens": 200
34.    }
35.
36.    # 调用 API
37.    try:
38.        response = requests.post(
39.            'https://api.deepseek.com/chat/completions',
40.            headers=headers,
41.            json=data,
42.            timeout=10
43.        )
```

```

44.
45.         if response.status_code == 200:
46.             return response.json()['choices'][0]['message']['content'].
strip()
47.         else:
48.             print(f"API 错误: {response.status_code}")
49.             return "抱歉, 暂时无法回答这个问题。"
50.
51.     except Exception as e:
52.         print(f"请求异常: {e}")
53.         return "网络错误, 请稍后重试。"

```

这个生成器的实现思路基于现代大语言模型的对话能力：

- 组织上下文信息：将检索到的相关文档整理成结构化的上下文。
- 构建智能提示：设计合理的提示词模板，引导大语言模型基于给定信息回答问题。
- 调用 API 服务：通过 HTTP 请求调用云端的大语言模型服务。
- 处理响应结果：解析 API 返回的数据，提取和清理生成的答案。
- 异常处理：处理网络错误、API 错误等异常情况，提供友好的错误提示。

4. RAG 系统的程序入口

程序清单 5.4 是整个 RAG 系统的程序入口，它不实现具体的功能算法，而是负责将知识库、检索器和生成器这三个核心组件有机地组合在一起，形成一个完整的、可工作的 RAG 系统。

程序清单 5.4 main.py。

```

1.  # main.py
2.  # 主程序 - 集成 RAG 流程
3.
4.  from dotenv import load_dotenv
5.  import os
6.
7.  # 导入模块
8.  from knowledge import documents
9.  from retriever import encode_documents, retrieve
10. from generator import generate_answer
11.
12. # 加载环境变量
13. load_dotenv()
14.
15.
16. def rag_system(question):
17.     """完整的 RAG 系统流程"""

```

```
18.     # 编码知识库文档
19.     doc_embeddings = encode_documents(documents)
20.
21.     # 检索相关文档
22.     relevant_docs = retrieve(question, documents, doc_embeddings)
23.     print(f"检索到的相关文档: {relevant_docs}")
24.
25.     # 生成答案
26.     answer = generate_answer(question, relevant_docs)
27.
28.     return answer, relevant_docs
29.
30.
31. # 使用示例
32. if __name__ == "__main__":
33.     question = "中国茶叶分为哪些种类?"
34.     answer, sources = rag_system(question)
35.
36.     print(f"\n 问题: {question}")
37.     print(f"回答: {answer}")
38.     print("\n 参考内容:")
39.     for idx, doc in enumerate(sources):
40.         print(f"[{idx + 1}] {doc}")
```

程序清单 5.1 至 5.4 完整呈现了 RAG（检索增强生成）的工作流程定义。

- 程序清单 5.1 构建了结构化知识库，作为系统的信息来源。
- 程序清单 5.2 实现了语义检索器，通过向量相似度匹配从知识库中定位相关内容。
- 程序清单 5.3 构成了生成器，依据检索结果构造提示词并调用大语言模型生成答案。
- 程序清单 5.4 作为主程序，协调上述模块依次执行，完成从查询到可追溯答案的端到端流程。

系统运行结果如下：

```
(llm-course) PS D:\llm-course\chapter05\simple-rag> python .\main.py
```

```
检索到的相关文档: ['中医药是中国传统医学，包括中药、针灸、推拿等治疗方法，强调阴阳平衡和整体观念。', '中国茶文化源远流长，茶叶分类包括绿茶、红茶、乌龙茶、黑茶、白茶、黄茶六大类。']
```

```
问题: 中国茶叶分为哪些种类?
```

```
回答: 中国茶叶主要分为六大类:
```

1. **绿茶**（如龙井、碧螺春）——不发酵，保留天然成分，口感清新。
2. **红茶**（如祁门红茶、滇红）——全发酵，茶汤红亮，滋味醇厚。

3. ****乌龙茶****（如铁观音、大红袍）——半发酵，兼具绿茶清香与红茶甘醇。
4. ****黑茶****（如普洱茶、安化黑茶）——后发酵，可长期存放，口感醇和。
5. ****白茶****（如白毫银针、寿眉）——微发酵，制作简约，茶味淡雅。
6. ****黄茶****（如君山银针、蒙顶黄芽）——轻发酵，有独特“闷黄”工艺，滋味甘润。

此外，还有再加工茶类（如花茶、紧压茶）等衍生品种，

参考内容:

- [1] 中医药是中国传统医学，包括中药、针灸、推拿等治疗方法，强调阴阳平衡和整体观念。
- [2] 中国茶文化源远流长，茶叶分类包括绿茶、红茶、乌龙茶、黑茶、白茶、黄茶六大类。

上面的运行结果表明：系统先检索到两条相关文档（中医药和中国茶文化），然后针对“中国茶叶分类”问题，基于检索到的茶文化信息生成了详细回答，列出了绿茶、红茶、乌龙茶、黑茶、白茶、黄茶六大类及其特点，并在最后提供了参考来源。

5.2 向量数据库的核心概念与技术

在 5.1 节中，我们实现了一个基于内存向量的简易 RAG 系统，它虽能进行语义检索，却暴露出内存方案的固有局限：数据无法持久化、容量受限于内存、检索效率随规模增长而下降。

向量数据库正是为解决这些工程化挑战而生的专业存储。它能够持久、高效地管理海量向量，通过专用索引实现快速语义检索，从而为 RAG 系统提供可靠且可扩展的“外部记忆体”。从内存原型到生产系统，引入向量数据库是关键一步。

本节将深入介绍向量数据库的核心原理、与传统数据库的区别以及主流技术选型，为后续集成应用奠定基础。

5.2.1 从传统数据库到向量数据库

传统数据库，无论是关系型数据库如 MySQL，还是 NoSQL 数据库如 MongoDB，其核心优势在于处理结构化数据。它们通过索引、查询语言（如 SQL）和事务机制（ACID）实现了对数据的精确存储、检索和管理。当查询条件明确时，例如查找“ID 为 123 的用户”或“状态为‘已完成’的订单”，传统数据库表现出极高的效率和可靠性。

然而，在面对非结构化数据，如长篇文本、图片或音频时，传统数据库的局限性便暴露无遗。比如：当用户查询“AI 的发展前景”时，传统数据库很难找到标题为“人工智能的未来”的文章，因为它们在字面上完全不匹配。这种基于字面匹配的检索方式，在需要理解字或词所包含内在含义也就是语义的场景时，传统数据库遇到了极大的挑战，面临着巨大的“语义鸿沟”。这也正是语义检索的痛点。

为了解决语义问题，需要一种能够表示“意思”的数据结构，这就是向量。

简单来说，向量是一个由数字组成的有序列表（例如 $[0.1, -0.5, 0.8, \dots]$ ）。在现代人工智能技术中，我们可以通过一种叫做嵌入模型的深度学习模型，将任何类型的数据——文本、图片、声音、甚至视频——转换成一个高维向量。

这个向量的神奇之处在于，它在数学空间中的位置和方向，能够编码原始数据的语义信息。语义上越相似的内容，其对应的向量在空间中的距离就越近。这个过程被称为“万物皆可向量化”。

有了表示语义的向量，我们如何衡量它们之间的相似程度呢？最常用的方法是余弦相似度。

想象一下，每个向量都是从坐标系原点出发的一条有向线段。余弦相似度计算的是这两条线段之间夹角的余弦值。

如果两个向量指向完全相同的方向（夹角为 0° ），余弦值为 1 ，表示它们完全相似。

如果两个向量正交（夹角为 90° ），余弦值为 0 ，表示它们毫不相关。

如果两个向量指向相反的方向（夹角为 180° ），余弦值为 -1 ，表示它们语义完全相反。

通过计算查询向量与数据库中所有向量的余弦相似度，我们就能找到与查询语义最接近的内容，从而实现真正的语义检索。

向量数据库的出现并非为了取代传统数据库，而是作为一种强大的补充，专门用于跨越这道语义鸿沟。它不关心数据是否结构化，也不进行精确匹配，其唯一的核心任务是：基于“相似度”进行检索。它回答的问题不再是“数据是否相等”，而是“数据在意义上是否相近”。这使得处理模糊、概念性的查询成为可能，为 AI 应用提供了前所未有的数据检索能力。

5.2.2 为什么 RAG 需要向量数据库

理解了向量数据库的基本原理后，我们来看看它在 RAG 架构中的核心价值。

1. 作为大模型外部记忆的核心组件

大语言模型虽然强大，但其“记忆”受限于两个因素：

训练数据截止日期：模型的知识停留在其训练完成的那一天，无法获取之后发生的新信息。

上下文窗口限制：模型单次处理文本的长度是有限的（例如几千到几万个 Token），无法将整个企业知识库都放入上下文中。

向量数据库完美地解决了这两个问题。它充当了 LLM 的外部记忆体或长期记忆。私有知识（如公司文档、产品手册、会议纪要）被预先向量化并存入数据库。当用户提问时，我们首先从向量数据库中检索出最相关的知识片段，然后将这些片段连同用户问题一起提交给 LLM 进行回答。这样，LLM 就能基于最新、最相关的私有知识来生成答案。

2. 实现高效、可扩展的语义检索

向量数据库是专门为大规模向量相似度搜索而优化的系统。它们内部实现了高效的索引算法（如 HNSW、IVF），能够在数百万甚至数十亿的向量中，毫秒级地完成相似度查询。这种高性能和可扩展性，是构建生产级 RAG 应用的关键保障。

5.2.2 向量数据库的基本原理

向量数据库的背后是一套清晰且高效的数学与工程原理。其核心工作流程可以概括为三步：嵌入、索引和查询。

1. 嵌入：从数据到向量

任何非结构化数据（文本、图片等）首先需要通过一个称为“嵌入模型”的 AI 模型进行转换。该模型将数据的高维语义信息压缩成一个固定长度的浮点数数组，即向量。这个过程可以理解为为每一段数据在“意义空间”中确定一个唯一的坐标。例如，“国王”和“女王”的向量坐标会非常接近，而它们与“香蕉”的向量坐标则会相距甚远。

2. 索引：构建高效的“相似度地图”

当向量数量达到百万、千万甚至亿级时，实时计算一个查询向量与所有库中向量的距离（即“暴力搜索”）会变得极其缓慢。为了解决性能问题，向量数据库采用了一种称为近似最近邻索引的技术。ANN 算法（如 HNSW、IVF）会预先对向量进行特殊的组织和聚类，构建一个高效的搜索索引。这个索引就像一张“相似度地图”，能够引导查询快速定位到最可能的邻近区域，从而在毫秒级时间内返回结果，同时牺牲微乎其微的准确性。

3. 查询：语义匹配的闭环

当一个查询请求（如一句话）到来时，系统会使用同一个嵌入模型将其转换为查询向量。随后，向量数据库利用构建好的 ANN 索引，快速找到与该查询向量最相似的一组向量。最后，数据库返回这些向量所对应的原始数据及其元数据，完成一次基于语义的精准检索。

5.2.3 主流向量数据库生态概览

向量数据库领域发展迅速，已形成功能定位清晰、适用于不同场景的丰富生态系统。根据部署形态、核心功能与目标场景，可将其大致分为三类：轻量级与嵌入式数据库、分布式生产级数据库，以及传统数据库的向量扩展。

1. 轻量级与嵌入式数据库

轻量级与嵌入式数据库专注于降低使用门槛，非常适合学习、算法验证、原型开发以及对数据规模和并发要求不高的应用。它们通常易于安装、API 简洁，且资源占用低。

ChromaDB: 一个开源的、AI 原生的嵌入式向量数据库。其核心优势在于极简的设计理念，内置了入门所需的一切，API 友好，并能与 LangChain、LlamaIndex 等 RAG 框架无缝集成，是快速构建和验证 LLM 应用的首选工具。目前仅支持 CPU 计算，通过乘积量化等技术优化存储与检索效率。

Milvus Lite: 作为全功能分布式向量数据库 Milvus 的轻量级版本，它可以在笔记本或容器中单机运行。其最大的价值在于与 Milvus 的 API 完全兼容，为开发者提供了从本地开发到生产级部署的平滑、无缝迁移路径。

FAISS (Facebook AI Similarity Search): 由 Meta 开发的高性能向量检索引擎库。它本身并非一个完整的数据库服务，而专注于极致的近似最近邻 (ANN) 搜索性能，提供多种索引算法并支持 GPU 加速，尤其适合离线批量处理和需要自定义检索逻辑的大规模数据集搜索场景。

SQLiteVec / DuckDB: 基于成熟轻量级数据库（如 SQLite、DuckDB）的向量扩展方案。它们为零部署成本的本地向量存储与分析提供了可能，适合将向量能力嵌入到已有的本地化应用 workflows 中。

2. 分布式生产级数据库

当应用需要处理海量数据（亿级甚至千亿级向量）、支撑高并发查询（百万级 QPS）并保证高可用性时，分布式向量数据库是必然选择。

Milvus: 最流行的开源分布式向量数据库之一。它采用云原生架构，具备水平扩展能力，支持 HNSW、IVF、PQ、DiskANN 等多种索引算法和 GPU 加速，能够在十亿级向量规模下实现毫秒级检索延迟。其开源社区活跃，是企业级 AI 应用（如推荐系统、图片搜索、RAG）的基石。

Weaviate: 一个功能强大的 AI 原生开源向量数据库。其特色在于模块化设计，内置了多种嵌入模型和向量化模块，并支持 GraphQL 查询语言。它超越了单纯的向量存储，允许开发者定义复杂的 Schema，将实体关联起来构建知识图谱，更擅长处理复杂的语义检索任务。

Qdrant: 使用 Rust 语言编写的开源向量数据库，以性能、内存安全和资源效率著称。它提供了丰富的 API 和过滤功能，强调易于部署与使用，同时不失企业级应用的稳定性与扩展性。

Pinecone: 一款全托管的商业向量数据库服务。用户无需关心任何底层架构和运维，即可享受企业级的性能、自动扩缩容和高可用性保障。它提供了完善的合规性认证（如 SOC 2, GDPR, HIPAA），非常适合希望专注于业务逻辑而非技术细节的企业用户。

3. 传统数据库的向量扩展

为了将向量检索能力与现有的事务性或分析型数据系统融合，主流数据库厂商纷纷推出

了向量扩展或内置功能。

PGVector: PostgreSQL 的开源向量扩展，允许用户在熟悉的 PostgreSQL 数据库中存储和查询向量。用户可以受益于 PostgreSQL 强大的 ACID 事务、JOIN 操作、成熟生态和运维工具，实现结构化数据与向量数据的统一管理。

Elasticsearch: 作为顶级的分布式搜索与分析引擎，Elasticsearch 内置了对向量的支持。它特别适合需要将传统全文搜索与向量语义搜索相结合的混合检索场景。

MongoDB Atlas Vector Search: MongoDB 云服务提供的向量搜索功能，让开发者可以在其文档数据库中直接进行向量相似性查询，简化了需要同时处理元数据和向量内容的应用开发。

Redis Stack (Tair): 基于内存的数据库 Redis 提供的向量检索模块（或阿里云 Tair 的向量能力），能够利用其极高的读写性能实现超低延迟的向量搜索，适用于对实时性要求极高的场景，如实时推荐。

5.3 向量数据库 Chroma 的使用

在前面介绍主流的向量数据库中，Chroma 作为开源的人工智能应用数据库，通过使知识、事实和技能能够嵌入 LLM，使构建 LLM 应用变得简单，在 LLM 应用开发应用较为广泛，也特别适合初学者。本节将介绍。本节将主要介绍 Chroma 向量数据库的使用，包括对向量数据的增删改查（CRUD）等核心操作。

源代码：

https://gitee.com/wu-solo_admin_admin/llm-course/tree/master/chapter05/chroma-crud

5.3.1 ChromaDB 的安装与设置

使用 Chroma 非常简单。

首先需要通过 pip 安装 Chroma。

```
pip install chromadb
```

安装完成后，即可在 Python 代码中导入并创建 Chroma 客户端。Chroma 支持两种运行模式：

- **内存模式:** 数据仅存储在内存中，程序关闭后数据丢失。适合快速测试和原型开发。
- **持久化模式:** 数据以文件形式保存在本地磁盘，程序重启后数据依然存在。适合绝大多数应用场景。

下面的程序创建了 Chroma 持久化客户端。

```
1. import chromadb
2.
3. # 创建一个持久化客户端，数据将保存在当前目录下的“chroma_db”文件夹中
4. client = chromadb.PersistentClient(path="./chroma_db")
```

```

5.
6. # 如果想使用内存模式，可以使用：
7. # client = chromadb.Client()

```

客户端创建后，下一步是创建或获取一个“集合”，它相当于传统数据库中的“表”。

```

1. # 获取或创建一个名为 'my_documents' 的集合
2. collection = client.get_or_create_collection(name="my_documents")

```

向量数据库的核心操作围绕着“集合”展开，比如上面的 `my_documents` 就是一个集合，类似于关系型数据库中的数据表。

5.3.2 核心概念：集合与文档

在深入操作之前，理解 Chroma 的几个核心概念至关重要：

- 集合：文档的容器，类似于关系型数据库中的表。一个集合中的文档通常共享同一个嵌入模型。
- 文档：想要存储和检索的原始内容，通常是一段文本。
- ID：每个文档的唯一标识符，用于精确地定位、更新或删除文档。
- 元数据：与文档关联的键值对信息，例如 `{"source": "user-manual.pdf", "page": 15}`。元数据本身不参与向量化，但可以用于查询时的精确过滤。
- 嵌入：文档的向量表示。Chroma 的便利之处在于，如果不指定嵌入模型，它会自动使用一个默认的轻量级模型来处理文本，这对初学者非常友好。

5.3.3 核心操作：数据的增删改查（CRUD）

源代码：\llm-course\chapter05\chroma-crud

Chroma 提供了直观的 API 来执行对集合中数据的 CRUD 操作。

1. 创建 `add()`

向集合中添加文档时，需要提供文档列表、对应的 ID 列表，以及可选的元数据列表。这三个列表的长度必须相同。

```

1. collection.add(
2.     documents=[
3.         "ChromaDB 是一个开源的向量数据库。",
4.         "它非常适合用于构建 RAG 应用。",
5.         "RAG 是检索增强生成的缩写。"
6.     ],

```

```
7.     ids=["doc1", "doc2", "doc3"],
8.     metadatas=[
9.         {"source": "intro-doc"},
10.        {"source": "intro-doc"},
11.        {"source": "glossary-doc"}
12.    ]
13. )
14.
15. # 检查集合中的文档数量
16. print(f"集合中现有文档数量: {collection.count()}")
17. # 输出: 集合中现有文档数量: 3
```

2. 读取 get()

可以通过 ID 精确地获取一个或多个文档。

```
1. result = collection.get(
2.     ids=["doc1", "doc3"]
3. )
4.
5. print(result)
6. # 输出:
7. # {'ids': ['doc1', 'doc3'],
8. #  'documents': ['ChromaDB 是一个开源的向量数据库。', 'RAG 是检索增强生成的缩写。'],
9. #  'metadatas': [{'source': 'intro-doc'}, {'source': 'glossary-doc'}]}
```

3. 更新 update()

要更新一个文档，需要提供其 ID 以及新的内容或元数据。

```
1. collection.update(
2.     ids=["doc2"],
3.     documents=["它非常适合用于构建基于大语言模型的 RAG 应用。"], # 更新文档内容
4.     metadatas=[{"source": "intro-doc", "updated": True}]          # 更新元数据
5. )
6.
7. # 再次获取验证
8. updated_doc = collection.get(ids=["doc2"])
9. print(updated_doc['documents'][0])
10. # 输出: 它非常适合用于构建基于大语言模型的 RAG 应用。
```

4. 删除 delete()

删除操作需要提供要删除的文档的 ID 列表。出于安全考虑，Chroma 不允许在不指定任

何条件的情况下清空整个集合。

```
1. collection.delete(  
2.     ids=["doc3"]  
3. )  
4.  
5. print(f"删除后集合中现有文档数量: {collection.count()}")  
6. # 输出: 删除后集合中现有文档数量: 2
```

5.3.4 查询与过滤

查询是向量数据库最核心的功能，Chroma 支持强大的语义查询和元数据过滤。

1. 语义查询

使用 `query()` 方法，并提供查询文本，Chroma 会自动将其向量化，并返回最相似的文档。

```
1. query_results = collection.query(  
2.     query_texts=["什么是 RAG?"],  
3.     n_results=2 # 返回最相似的前 2 个结果  
4. )  
5.  
6. print(query_results)  
7. # 输出将包含 'documents', 'distances', 'metadatas' 等  
8. # 'distances' 值越小，代表相似度越高
```

2. 元数据过滤

可以在查询时使用 `where` 子句，对元数据进行精确过滤，实现语义搜索与布尔查询的结合。

```
1. # 查询与“向量数据库”相关，且来源为“intro-doc”的文档  
2. filtered_results = collection.query(  
3.     query_texts=["向量数据库"],  
4.     where={"source": "intro-doc"}  
5. )  
6.  
7. print(filtered_results)
```

Chroma 支持丰富的过滤操作符，如 `$eq` (等于), `$ne` (不等于), `$in` (包含于列表), `$gt` (大于) 等，为构建复杂的查询逻辑提供了强大支持。

5.3.5 RAG 使用 Chroma 向量数据库

上一节介绍的基础 RAG 系统虽然在概念上清晰易懂，但在实际应用中存在几个关键限制。首先，内存中的向量计算方式无法处理大规模知识库；其次，每次程序重启都需要重新计算所有文档的嵌入向量，效率低下；最后，缺乏专业向量检索算法支持，检索性能受限。为解决这些问题，实际项目通常需要引入专门的向量数据库。

本节将在 simple-rag 这个比较简单而又基础的 RAG 系统之上，引入 Chroma 向量数据库来持久化存储知识库，实现一个更健壮的 RAG 系统。系统整体架构与基础 RAG 系统保持一致，主要改动在于使用 Chroma 向量数据库重构了检索器模块，并相应调整了主程序入口。项目结构如下：

```
simple-rag-chroma/
├── main.py      # 主程序，集成 RAG 流程（有改动）
├── knowledge.py # 知识库模块，存储文档数据
├── retriever.py  # 检索模块，使用 Chroma 向量数据库进行语义检索（有改动）
├── generator.py  # 生成模块，调用 API 生成答案
├── .env          # 环境变量配置文件
├── README.md    # 项目说明文档
└── chroma_db/   # Chroma 向量数据库持久化存储目录
```

其中改动的地方是检索器模块（retriever.py）和主程序入口文件（main.py），下面重点介绍这两个模块文件。

源码：

https://gitee.com/wu-solo_admin_admin/llm-course/tree/master/chapter05/simple-rag-chroma

1. 基于 Chroma 的检索器模块

检索器模块在程序清单 5.6 中进行了重要改进。程序清单 5.2 中的版本为了简化复杂性，采用了内存计算和函数式设计，适合小规模演示。而程序清单 5.6 则引入 Chroma 向量数据库，实现了高效检索与知识积累，为生产环境应用奠定了基础。

程序清单 5.6 retriever.py。

```
1.# retriever.py
2.# 语义检索模块 - 使用 Chroma 向量数据库
3.
4.from sentence_transformers import SentenceTransformer
5.import chromadb
6.
7.# 初始化嵌入模型
8.embedding_model = SentenceTransformer('all-MiniLM-L6-v2')
9.
10.
11.def init_chroma_collection(collection_name="knowledge_base"):
12.    """初始化 Chroma 集合"""
```

```
13. # 创建 Chroma 客户端（新 API）
14. client = chromadb.PersistentClient(path="./chroma_db")
15.
16. # 创建或获取集合
17. collection = client.get_or_create_collection(
18.     name=collection_name,
19.     metadata={"hnsw:space": "cosine"}
20. )
21. return collection
22.
23.
24. def encode_documents(documents, collection_name="knowledge_base"):
25.     """将文档添加到 Chroma 向量数据库"""
26.     collection = init_chroma_collection(collection_name)
27.
28.     # 检查集合是否已有数据
29.     if collection.count() == 0:
30.         # 添加文档到 Chroma
31.         ids = [f"doc_{i}" for i in range(len(documents))]
32.         collection.add(
33.             documents=documents,
34.             ids=ids
35.         )
36.
37.     return collection
38.
39.
40. def retrieve(query, collection, top_k=2):
41.     """从 Chroma 向量数据库检索相关文档"""
42.     # 执行语义搜索
43.     results = collection.query(
44.         query_texts=[query],
45.         n_results=top_k
46.     )
47.
48.     # 返回相关文档
49.     return results['documents'][0]
```

代码说明：

该模块基于 chromadb 实现向量数据库的持久化存储与检索，主要包含三个函数：

- `init_chroma_collection`：创建持久化客户端（存储目录为 `./chroma_db`），获取或创建指定名称的集合，并通过元数据设置相似度度量（此处为余弦相似度），最后返回集合对象。

- `encode_documents`: 接收文档列表，初始化集合，并在集合为空时生成唯一 ID（如 `doc_0`），调用 `add` 方法自动将文档向量化并存入数据库。
- `retrieve`: 接收查询文本、集合对象和返回文档数量 `top_k`，调用 `query` 方法执行相似度搜索，返回最相似的文档列表。

使用流程：首先调用 `encode_documents` 将文档存入 Chroma 集合，然后通过 `retrieve` 执行相似度搜索。所有数据持久化存储在 `./chroma_db` 目录，相似度度量可配置为 `cosine`（余弦）、`l2`（欧氏距离）或 `ip`（内积）。

2. 主程序模块

项目的主程序入口 `main.py` 通过引入上述检索器函数进行了重构，将知识库初始化从每次调用改为一次性执行，大幅提升了运行效率。代码结构更加清晰，实现了初始化与检索逻辑的分离，并优化了输出提示，使系统更模块化。

程序清单 5.7 `main.py`。

```
1. # 导入模块
2. from knowledge import documents
3. from retriever import encode_documents, retrieve
4. from generator import generate_answer
5.
6. # 加载环境变量
7. load_dotenv()
8.
9. def rag_system(question):
10.     """完整的 RAG 系统流程"""
11.     # 初始化 Chroma 集合并添加文档
12.     collection = encode_documents(documents)
13.     # 从 Chroma 检索相关文档
14.     relevant_docs = retrieve(question, collection)
15.     print(f"检索到的相关文档: {relevant_docs}")
16.     # 生成答案
17.     answer = generate_answer(question, relevant_docs)
18.     return answer, relevant_docs
19.
20. # 使用示例
21. if __name__ == "__main__":
22.     question = "中国茶叶分为哪些种类? "
23.     answer, sources = rag_system(question)
24.     print(f"\n 问题: {question}")
25.     print(f"回答: {answer}")
26.     print("\n 参考内容:")
27.     for idx, doc in enumerate(sources):
28.         print(f"[{idx + 1}] {doc}")
```


代码说明：

主程序模块实现了完整的 RAG 系统流程：

- 导入模块：从 `knowledge` 导入文档数据；从 `retriever` 导入向量数据库操作函数；从 `generator` 导入答案生成函数；并导入 `dotenv` 用于加载环境变量。
- 核心流程：
 - `rag_system` 函数封装了完整的 RAG 流程，包括文档编码（持久化）、语义检索和答案生成。
 - 程序的主入口点，用于演示完整的 RAG 系统功能。通过定义测试问题，调用 RAG 系统生成答案，并格式化输出问题、答案和参考文档，展示系统的实际运行效果。
 - `question` 定义测试问题，作为 RAG 系统的输入查询。
 - `answer, sources` 是调用 `rag_system` 函数处理问题，返回两个结果，前者大语言模型生成的答案，后者检索到的相关文档列表
 - `for` 语句使用 `enumerate` 函数遍历检索到的来源文档列表，`idx` 是文档索引，`doc` 是文档内容。
 - 最后一句格式化打印每个来源文档，使用编号（从 1 开始）和文档内容。最终格式化输出问题、答案和参考文档，展示完整的问答过程。

运行该系统，其输出结果与 5.1.3 节构建的基础 RAG 系统运行结果一致，此处不再赘述。

5.3.6 Chroma 支持的嵌入模型

如果说向量数据库是 RAG 系统的“记忆图书馆”，那么嵌入模型就是将所有知识（无论是文字、图片还是声音）翻译成图书馆唯一通用语言——“向量语”的“超级翻译官”。这个翻译官的水平，直接决定了图书馆管理员（检索系统）能否快速、准确地找到用户想要的书籍。因此，选择和优化嵌入模型是构建高质量 RAG 系统的基石。

1. 嵌入模型与向量数据库、RAG 的协同关系

嵌入模型、向量数据库和 RAG 三者之间并非完全意义上缺一不可的关系，但它们紧密协作，共同构建起高效的信息检索与生成系统。

嵌入模型与向量数据库（这里以 Chroma 为例说明，但原理适用于各类向量数据库）之间存在类似生产者与仓库的协作关系。嵌入模型扮演着“向量生产者”的角色，它具备将原始数据转化为向量表示的能力。原始文档，无论是 PDF、网页还是其他文本形式，经过嵌入模型处理后，都会被转换成一个具有特定语义特征的向量。而向量数据库 Chroma 则如同“向量仓库”，它为这些生成的向量提供了存储和管理空间。在入库流程中，我们利用嵌入模型将原始文档转化为向量，随后将 <向量, 原始文档> 这一数据对存入 Chroma 数据库中。

在查询阶段，当用户提出一个问题时，同样需要使用与入库时相同的嵌入模型将问题也

转换成一个向量。这是因为只有使用同一个嵌入模型，才能保证问题向量和文档向量处于同一语义空间，具备可比性。如果入库和查询使用不同的嵌入模型，就如同两个人说着不同的语言，无法进行有效的交流和比较，比较结果也就失去了意义。**Chroma** 数据库会利用这个查询向量，在仓库中通过特定的算法（如计算向量之间的相似度，距离越近表示相似度越高）寻找与之最相似的文档向量，进而找到对应的原始文档。

嵌入模型和向量数据库的组合，共同构成了 **RAG** 架构中的“**R**”，即检索环节。**RAG** 架构旨在通过检索外部知识来增强大语言模型（**LLM**）的回答能力，使其生成的结果更具知识性和准确性。而检索环节作为整个架构的前端，负责从海量数据中快速精准地找到与用户问题相关的信息，嵌入模型和向量数据库的协同工作正是实现这一目标的关键。

在 **RAG** 中的工作流中，嵌入模型和向量数据库协同一起工作。具体是：

在检索阶段，当用户提出问题后，首先由嵌入模型将问题转化为向量形式。这个向量作为查询条件，被输入到向量数据库 **Chroma** 中。**Chroma** 数据库根据向量的相似度算法，在已存储的文档向量中搜索与之最匹配的向量，进而找到对应的原始文档。这一过程实现了从用户问题到相关文档的精准定位。

在增强阶段，检索到的相关文档并非直接用于生成答案，而是与用户原始问题进行拼接。这种拼接操作形成了一个信息更为丰富的提示（**Prompt**），它不仅包含了用户的核心问题，还融入了从外部知识库中获取的相关背景信息，为大语言模型提供了更全面的上下文。

在生成阶段，将增强后的提示输入到大语言模型（**LLM**）中。大语言模型基于这些真实、准确的上下文信息，运用自身的语言生成能力，生成最终的答案。由于有了外部知识的支撑，生成的答案在准确性和可靠性上得到了显著提升。

虽然理论上存在其他方式可以实现类似 **RAG** 的功能，但嵌入模型和向量数据库的组合为 **RAG** 系统提供了高效、可靠的检索基础，是实现“知识增强”和“事实依据”的重要支撑。没有它们，**RAG** 系统在检索相关知识和提供准确上下文方面将面临巨大挑战，难以达到预期的效果。不过，这并不意味着它们是不可替代的唯一选择，随着技术的不断发展，可能会有新的技术和方法出现，但在当前的技术体系下，嵌入模型与向量数据库在 **RAG** 架构中发挥着不可替代的重要作用。

2. Chroma 支持的嵌入模型

Chroma 的一个核心优势在于其灵活性，它并不绑定任何单一的嵌入模型。相反，它通过一个标准化的 `embedding_function` 接口，允许你自由选择 and 集成最适合你需求的模型。这就像一个“万能插头”，可以连接各种“电器”（嵌入模型）。

表 5.2 Chroma 支持的嵌入模型

模型名称	模型大小
all-mpnet-base-v2	420 MB
multi-qa-mpnet-base-dot-V1	420 MB

all-distilroberta-V1	290 MB
all-MiniLM-L12-v2	120 MB
multi-qa-distilbert-cos-V1	250 MB
all-MiniLM-L6-v2	80 MB
multi-qa-MiniLM-L6-cos-V1	80 MB
paraphrase-multilingual-mpnet-base-v2	970 MB
paraphrase-albert-small-v2	43 MB
paraphrase-multilingual-MiniLM-L12-v2	420 MB
paraphrase-MiniLM-L3-v2	61 MB
distiluse-base-multilingual-cased-v1	480 MB
distiluse-base-multilingual-cased-v2	480 MB

下面我们将 **Chroma** 支持的模型分为四大类，并逐一介绍其用法、优缺点和适用场景。

(1) Chroma 内置默认模型

这是最简单的入门方式，几乎不需要任何配置。

模型介绍：**Chroma** 默认使用 **all-MiniLM-L6-v2**，这是一个由 **Sentence Transformers** 提供的轻量级、高质量的英文模型。它在速度和性能之间取得了很好的平衡。

如何使用：在创建或连接 **Chroma** 集合时，不指定 **embedding_function** 参数即可。

```

1.     import chromadb
2.     from langchain_community.vectorstores import Chroma
3.     from langchain_core.documents import Document
4.
5.     # Chroma 会自动使用默认的嵌入模型
6.     client = chromadb.Client()
7.     collection = client.create_collection("my_collection")
8.
9.     # 在 LangChain 中也一样
10.    vectorstore = Chroma.from_documents(
11.        documents=[Document(page_content="Hello world")],
12.        # embedding_function=... # 不指定，就用默认的
13.        persist_directory="./default_db"
14.    )

```

优点：

零配置：无需安装额外模型或设置 **API** 密钥，开箱即用。

快速启动：非常适合快速原型验证和学习。

缺点：

仅支持英文：对中文等非英语语言的支持很差。

性能中等：虽然不错，但通常不如专门为特定任务或语言训练的模型。

适用场景：

快速体验 Chroma 和 RAG 流程。

英文环境下的概念验证项目。

(2) Hugging Face 模型（推荐）

这是最常用、最灵活的选择，特别是对于中文应用场景。

模型介绍：Hugging Face 是世界上最大的开源模型社区，提供了成千上万个预训练的嵌入模型，涵盖多种语言和特定领域（如法律、医疗）。shibing624/text2vec-base-chinese 就是一个优秀的中文模型。

如何使用：通常通过 langchain-huggingface 包来集成。

```

1. import chromadb
2. from langchain_community.vectorstores import Chroma
3. from langchain_core.documents import Document
4. from langchain_huggingface import HuggingFaceEmbeddings
5.
6. # --- 关键修复：定义 documents 变量 ---
7. documents = [
8.     Document(page_content="LangChain 是一个强大的框架，用于构建基于大语言模型的应用。", metadata={"source": "langchain-doc"}),
9.     Document(page_content="它通过模块化的组件，如模型、提示、索引和链，简化了开发流程。", metadata={"source": "langchain-doc"}),
10.    Document(page_content="ChromaDB 是一个开源的向量数据库，非常适合用于 AI 应用的原型开发。", metadata={"source": "chroma-doc"}),
11. ]
12.
13. # 1. 实例化嵌入模型
14. embeddings = HuggingFaceEmbeddings(
15.     model_name="shibing624/text2vec-base-chinese", # 指定模型名称
16.     model_kwargs={'device': 'cpu'}, # 使用 CPU
17.     encode_kwargs={'normalize_embeddings': True} # 归一化向量，提高余弦相似度计算效果
18. )
19.
20. # 2. 将模型实例传递给 Chroma
21. # 现在，Python 知道 documents 是什么了
22. vectorstore = Chroma.from_documents(
23.     documents=documents, # <--- 这行现在可以正常工作了
24.     embedding=embeddings,
```

```

25.     persist_directory="./huggingface_db"
26. )
27.
28. print("向量数据库已成功创建并持久化!")

```

优点:

海量选择: 可以根据语言、性能、大小等需求自由选择模型。

强大的中文支持: 有大量高质量的中文开源模型。

本地部署: 数据无需离开本地环境, 完全保护隐私。

免费: 模型本身是免费的。

缺点:

首次下载慢: 第一次使用时需要下载模型文件, 可能耗时较长。

需要额外依赖: 需要安装 `sentence-transformers`、`torch` 等库。

适用场景:

生产级应用, 特别是涉及中文的应用。

对数据隐私有严格要求的项目。

需要针对特定领域优化的场景。

(3) API 提供商模型

通过调用第三方 API 来获取嵌入, 如 OpenAI、Cohere 等。

模型介绍: 这些通常是大型科技公司提供的最先进的嵌入模型, 如 OpenAI 的 `text-embedding-ada-002` 或 `text-embedding-3-small/large`。

如何使用: 通过相应的 LangChain 集成包。

如何使用: 通常通过 `langchain-huggingface` 包来集成。

```

1.  # --- 1. 导入所有必需的库 ---
2.  import os
3.  from langchain_openai import OpenAIEmbeddings
4.  from langchain_community.vectorstores import Chroma # <-- 关键修复: 导入 Chroma
5.  from langchain_core.documents import Document      # <-- 关键修复: 导入 Document
6.
7.  # --- 2. 设置 API 密钥 ---
8.  #  请将 "你的真实 OpenAI_API_Key" 替换为你的密钥
9.  os.environ["OPENAI_API_KEY"] = "你的真实 OpenAI_API_Key"
10.
11. # --- 3. 定义要处理的文档 ---
12. #  <-- 关键修复: 定义 documents 变量
13. documents = [
14.     Document(page_content="LangChain 是一个强大的框架, 用于构建基于大语言模型的应用。",
15.               metadata={"source": "langchain-doc"}),

```

```
15.     Document(page_content="它通过模块化的组件，如模型、提示、索引和链，简化了开发流程。", metadata={"source": "langchain-doc"}),
16.     Document(page_content="ChromaDB 是一个开源的向量数据库，非常适合用于 AI 应用的原型开发。", metadata={"source": "chroma-doc"}),
17. ]
18.
19. # --- 4. 实例化 OpenAI 嵌入模型 ---
20. embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
21.
22. # --- 5. 将模型和文档传递给 Chroma，创建向量数据库 ---
23. # 现在，Python 认识 Chroma 和 documents 了
24. vectorstore = Chroma.from_documents(
25.     documents=documents, # <-- 这行现在可以正常工作了
26.     embedding=embeddings,
27.     persist_directory="./openai_db"
28. )
29.
30. print("使用 OpenAI 嵌入的向量数据库已成功创建并持久化!")
```

优点：

性能顶尖：通常在通用基准测试中表现最佳。

无需本地资源：不需要强大的本地 GPU 或大量内存来运行模型。

维护简单：无需自己管理和更新模型。

缺点：

付费服务：按使用量收费，成本可能较高。

网络依赖：需要稳定的网络连接，存在延迟。

数据隐私：数据需要发送到第三方服务器进行处理。

适用场景：

对嵌入质量有极致要求，且预算充足的项目。

快速迭代，不想在本地环境配置上花费时间。

5.4 高级 RAG (Advanced RAG)

在前面的学习中，我们已经掌握了基础 RAG (Retrieval-Augmented Generation) 的基本实现方式，即通过“检索+生成”的流程，将外部知识引入大语言模型，从而提升回答的准确性。然而，在真实应用场景中，简单的 RAG 流程往往难以满足复杂需求，例如多轮问答、专业领域检索以及高可靠性要求等。为此，研究者提出了高级 RAG (Advanced RAG)，通过对各个环节进行系统性优化，显著提升整体性能。

本节将系统介绍高级 RAG 的概念与实现方法。

5.4.1 问题驱动：朴素 RAG 的局限

朴素 RAG 通常采用“索引→检索→生成”的线性流程，虽然结构简单，但在实际应用中存在以下三类典型问题：

（1）检索能力不足

用户查询往往具有口语化、模糊或不完整等特点，直接进行向量检索可能无法准确表达用户真实意图，导致召回结果偏离问题核心。同时，仅依赖向量相似度检索，容易引入冗余或噪声信息。

（2）生成质量不稳定

当检索结果不足或质量不高时，大语言模型可能产生“幻觉”（hallucination），即生成不基于事实的信息。此外，过多无关上下文也可能干扰模型判断，降低回答质量。

（3）系统适应性有限

朴素 RAG 通常面向单一数据源，缺乏对多种数据类型（如结构化数据、表格、知识图谱）的支持，也难以适应复杂查询（如多跳推理、跨领域问题）。

综上所述，朴素 RAG 难以满足复杂应用场景的需求，需要在各个环节引入更加精细化的优化策略。高级 RAG 正是在这一背景下提出的。

5.4.2 方法框架：高级 RAG 整体架构

与朴素 RAG 不同，高级 RAG 并非单一技术，而是一种面向系统优化的方法框架。它将原有流程拆解为多个可独立优化的模块，包括索引构建、查询处理、检索优化以及生成控制等环节。

为了更直观地理解高级 RAG 的整体架构及各优化策略之间的关系，图 5.7 展示了高级 RAG 的完整流程。



图 5.7 高级 RAG 流程图

图 5.7 展示了高级 RAG (Advanced RAG) 的整体处理流程。系统在索引阶段完成数据清洗、语义分块与向量化处理；在检索阶段通过查询改写、查询扩展、混合检索与重排序等技术提升检索质量；在生成阶段通过提示压缩、迭代检索与忠实度检查等机制优化输出结果。

从本质上看，高级 RAG 体现了一种“模块化 + 可优化”的系统设计思想，即通过在各阶段引入针对性优化策略，实现整体性能的协同提升。

在此基础上，下面将分别从索引、检索与生成三个阶段，对高级 RAG 的关键技术进行详细讲解。

5.4.3 高级 RAG 的索引阶段优化

索引阶段的核心目标是构建高质量知识库，为检索提供可靠基础。高级 RAG 在此阶段的优化措施包括：

1. 数据清洗与降噪

在将文档切分前和构建索引前，对原始数据进行清洗，去除 HTML 标签、特殊符号及无关内容，从源头减少噪声干扰。

2. 语义分块 (Semantic Chunking)

传统固定长度切分容易破坏语义连贯性。语义分块通过分析句子间的语义相似度，将内容划分为逻辑完整的信息单元，从而提升检索质量。

3. 元数据增强

为每个文本块添加结构化元数据（如标题、时间、类别等），在检索阶段可作为过滤条件，提高检索精度。例如在医疗问答中，可使用“疾病类型”作为筛选条件。

4. 混合索引构建

同时构建向量索引（密集检索）与关键词索引（稀疏检索），兼顾语义检索与精确匹配能力，为后续混合检索提供基础。

5.4.4 高级 RAG 的检索阶段优化

检索阶段决定系统“能够找到什么信息”，是 RAG 系统性能的关键。高级 RAG 在该阶段引入多种优化方法：

- 查询改写（Query Rewriting）：利用大语言模型对用户查询进行重写，使其表达更加清晰、规范，从而提高检索匹配度。
- 查询扩展（Query Expansion）：从多角度生成查询变体（同义表述、相关子问题），对每条查询进行检索后合并结果，提升召回率和答案全面性。
- 查询路由（Query Routing）：根据问题类型，将查询分配至不同数据源或检索策略，实现更精细化处理。例如：
 - ✓ 最新新闻 → 实时索引
 - ✓ 专业术语 → 领域专属索引
 - ✓ 高语义理解 → 向量索引
- 混合检索（Hybrid Retrieval）：结合向量检索（语义匹配）与关键词检索（精确匹配），综合提升检索效果。
- 结果重排序（Re-ranking）：使用交叉编码器（Cross-Encoder）或 API（如 Cohere 等）对检索结果进行精细排序，提高相关性和答案质量。

通过上述优化，高级 RAG 显著提升了召回率、结果相关性和噪声控制，为生成阶段提供高质量输入。

5.4.5 高级 RAG 的生成阶段优化

在获得高质量检索结果后，生成阶段负责将信息转化为最终答案。高级 RAG 在该阶段重点关注输出的准确性与可靠性：

- 提示压缩与过滤：对检索结果进行筛选与压缩，仅保留关键信息，避免上下文过长导致模型性能下降。
- 多轮生成与迭代检索
- ✓ 递归检索：如果生成阶段发现信息不足，可再次查询知识库补充内容。

- ✓ 迭代检索：对初步答案进行反思、验证，并再次检索补充，提高答案完整性。
- 忠实度检查与幻觉抑制
- ✓ 提示模型仅基于检索内容回答，避免自行编造。
- ✓ 可使用小模型或规则引擎进行答案验证，如发现偏差则提示重新生成或过滤。
- ✓ 答案多样性控制：当信息不足时，系统可再次发起检索，实现“检索—生成—再检索”的循环机制。

生成阶段优化依赖高质量检索输入，因此索引和检索优化是答案可信度的前提。

5.4.6 综合实现：高级 RAG 系统示例

在理解高级 RAG 各个优化策略后，下面通过一个综合示例项目 **advanced-rag**，展示如何在实际系统中将这些技术进行组合应用。

项目 **advanced-rag** 的结构如下：

```
advanced-rag/
├── advanced_rag.py      # 高级 RAG 主函数
├── chunking.py          # 文档分块功能
├── config.py            # 配置文件
├── evaluation.py        # 性能评估模块
├── generation.py        # 答案生成模块
├── knowledge_base.py    # 知识库管理模块
├── main.py              # 主程序入口
├── query_rewriting.py   # 查询改写模块
├── reranking.py         # 文档重排序模块
├── retrieval.py         # 检索模块
└── data/                # 数据目录（用于存储文档）
```

项目 **advanced-rag** 实现了如下功能：

- 查询改写：将用户查询改写为多个相关查询，提高检索准确率。
- 多路召回：结合向量检索和关键词检索，获取更全面的文档。
- 重排序：对检索结果进行重排序，提高相关性。
- 答案生成：基于检索到的文档生成高质量回答。
- 性能评估：支持评估 RAG 系统的性能指标。

整个项目为了聚焦检索优化部分，在生成阶段采用了从检索结果中提取匹配文档的方式，不依赖外部大语言模型，仍可完整演示高级 RAG 从查询到生成的各项核心功能。

源码：

https://gitee.com/wu-solo_admin_admin/llm-course/tree/master/chapter05/advanced-rag

下面重点介绍查询改写是如何实现的，其他模块可自行阅读源码学习。

1. 查询改写模块的设计思想

查询改写是高级 RAG 系统的重要优化技术，其核心思想是将用户的原始查询转换为多个相关的查询变体，从而提高检索的准确率和召回率。在本项目中，由于采用本地化实现，我们使用同义词扩展的方式生成查询变体：

(1) 问题分析：用户查询可能存在词汇多样性问题（如“报销”和“报账”表达相同意思），单一查询可能无法覆盖所有相关文档。

(2) 解决方案：通过预定义同义词词典，将原始查询中的关键词替换为其同义词，生成多个查询变体。

(3) 优化策略：

- 使用领域特定的同义词表（针对财务报销场景）。
- 保留原始查询，确保核心意图不变。
- 限制生成的查询数量（最多 3 个），避免过度扩展。
- 针对多关键词查询，采用独立替换策略，避免多次替换导致的语义混乱。

2. 查询改写模块的实现

程序清单 5.6 query_rewriting.py

```
1.# query_rewriting.py - 查询改写功能
2.def rewrite_query(original_query):
3.    """查询改写功能：生成多个优化后的查询（本地版本）"""
4.    print("使用本地查询改写...")
5.
6.    # 添加一些简单的同义词变体
7.    synonyms = {
8.        "报销": ["报账", "费用报销"],
9.        "出差": ["旅行", "外勤", "公干"],
10.        "交通费": ["交通费用", "路费", "车票"],
11.        "住宿费": ["住宿费用", "房费", "酒店费用"],
12.        "餐饮费": ["餐费", "伙食费", "餐饮补贴"],
13.        "发票": ["票据", "税票", "增值税发票"]
14.    }
15.
16.    variations = [original_query] # 始终保留原始查询
17.
18.    for key, syn_list in synonyms.items():
19.        if key in original_query:
20.            # 为每个同义词生成一个新查询（如果尚未存在）
21.            new_queries = [original_query.replace(key, syn) for syn in syn_list]
22.            variations.extend([q for q in new_queries if q not in variations])
23.
24.    return variations[:3]
25.
```

```

26.
27. # 测试代码
28. if __name__ == "__main__":
29.     # 测试查询改写功能
30.     test_queries = [
31.         "公司出差报销需要提供哪些费用发票？",
32.         "住宿费报销标准是多少？",
33.         "报销流程是怎样的？",
34.         "餐饮费补助标准是多少？"
35.     ]
36.
37.     for query in test_queries:
38.         print(f"\n 原始查询: {query}")
39.         rewritten_queries = rewrite_query(query)
40.         print(f"改写查询: {rewritten_queries}")

```

代码说明：

（1）函数定义：`rewrite_query` 接收原始查询字符串，返回改写后的查询列表（最多 3 个）。

（2）同义词词典：定义了财务报销领域的关键词及其同义词，如“报销”对应“报账”和“费用报销”。

（3）改写逻辑

- 初始化 `variations` 列表，包含原始查询。
- 遍历同义词词典，若原始查询中包含关键词，则使用列表推导式为每个同义词生成新查询。
- 通过 `extend` 将新查询加入列表，并用 `if q not in variations` 去重。
- 最后通过切片 `[:3]` 限制返回的查询数量，避免过多变体。

（4）测试代码：提供了多个测试用例，可直接运行验证模块功能。

3. 运行结果示例

执行 `python query_rewriting.py` 后，程序输出如下：

```
(llm-course) PS D:\llm-course\chapter05\advanced-rag> python query_rewriting.py
```

原始查询: 公司出差报销需要提供哪些费用发票？

使用本地查询改写...

改写查询: ['公司出差报销需要提供哪些费用发票？', '公司出差报账需要提供哪些费用发票？', '公司出差费用报销需要提供哪些费用发票？']

原始查询: 住宿费报销标准是多少？

使用本地查询改写...

改写查询: ['住宿费报销标准是多少?', '住宿费报账标准是多少?', '住宿费费用报销标准是多少?']

原始查询: 报销流程是怎样的?

使用本地查询改写...

改写查询: ['报销流程是怎样的?', '报账流程是怎样的?', '费用报销流程是怎样的?']

原始查询: 餐饮费补助标准是多少?

使用本地查询改写...

改写查询: ['餐饮费补助标准是多少?', '餐费补助标准是多少?', '伙食费补助标准是多少?']

从结果可以看出, 系统为每个原始查询生成了最多 3 个查询变体:

- 对于包含“报销”的查询, 生成了“报账”和“费用报销”两种变体。
- 对于“住宿费”, 生成了“住宿费报账”和“住宿费费用报销”等变体。
- 对于“餐饮费补助”, 将“餐饮费”替换为“餐费”和“伙食费”, 生成了相关变体。

这些查询变体将被用于多路召回, 通过向量检索和关键词检索获取更全面的相关文档, 从而提高回答的准确性和完整性。

5.5 RAG 系统的评估简介

构建 RAG 系统后, 科学评估其性能十分关键。全面评估体系能精准定位系统瓶颈, 为迭代优化指明方向。本节将系统介绍评估 RAG 系统的三大核心标准——忠实性、上下文相关性与答案相关性, 以及围绕这些标准开展评估的具体步骤。

5.5.1 RAG 系统的评估标准

RAG 系统主要涉及问题、检索上下文和 LLM 生成答案三个方面, 评估即衡量这三者间的相关程度, 具体有忠实性(答案与上下文事实一致性)、上下文相关性(检索上下文围绕问题且含关键信息)、答案相关性(答案直接、完整、无冗余地回答问题)。这三个标准对应信息检索准确性、生成可靠性与输出有效性, 是衡量系统质量的基石。

1. 忠实性

忠实性, 即事实一致性, 是评估 RAG 系统的首要指标, 衡量答案与检索上下文的事实相符程度。高度忠实性的答案需严格依据上下文, 不捏造、歪曲信息。评估时主要考察:

- 事实一致性: 答案事实断言能否在上下文中找到支撑。
- 幻觉抑制: 系统是否避免生成源于内部参数而非上下文的“幻觉”信息。
- 可信度保障: 忠实性决定输出可信度, 用户需信任答案基于知识源生成。

案例分析: 给定上下文“特斯拉 Model 3 续航里程 455 - 629 公里, 支持 V3 超级充电桩, 充电 15 分钟补充约 250 公里续航”。用户问“特斯拉 Model 3 充电速度怎么样?”, 正面案例回答“支持 V3 超充, 15 分钟补 250 公里续航”, 基于上下文, 体现忠实性; 反面案例

称“10 分钟充满 80%”，数据无依据，属“幻觉”，损害可信度。

2. 上下文相关性

上下文相关性评估检索阶段表现，关注检索上下文与用户问题高度相关且含核心信息。检索质量影响后续答案生成。评估时主要考察：

- 检索精度：返回上下文是否围绕问题核心意图，而非边缘信息。
- 信息完备性：上下文是否提供解决问题所需关键信息，避免片面或缺失。
- 噪声控制：检索结果是否混杂冗余信息，干扰生成模型判断。

案例分析：用户问“《蒙特利尔议定书》主要目标是什么？”，正面案例检索上下文直接给出“通过淘汰 ODS 保护臭氧层”，高度相关；反面案例提供签署时间、生效时间等背景信息，缺失关键目标，还有无关噪声，系统难生成正确答案。

3. 答案相关性

答案相关性从用户视角出发，评估答案是否有效回应问题，关注答案“效用”，即是否直接、完整、清晰且无冗余地解决问题。评估时需综合考量：

- 问题针对性：答案是否直接回答问题，避免答非所问。
- 答案完整性：答案是否覆盖问题所有方面，不遗漏要点。
- 表达简洁性：答案是否言简意赅，逻辑清晰，避免重复、模板化语言或无关细节。

案例分析：用户问“光合作用主要产物是什么？”，正面案例直接回答“有机物（如葡萄糖）和氧气”，相关性高；反面案例介绍光合作用过程、重要性及发生场所，不直接、不完整且冗余，相关性差。

忠实性、上下文相关性和答案相关性是评估 RAG 系统的“黄金三角”。上下文相关性确保输入质量，是基础；忠实性约束生成过程，是核心；答案相关性决定输出效用，是目标。实际评估需协同考察三者，全面准确判断系统性能，指引优化方向。

5.5.2 RAG 系统的评估步骤

对 RAG 系统科学、系统评估是优化性能的关键，规范流程能确保结果可比性与可重复性，为迭代优化提供指引。通常将评估过程提炼为构建评估数据集、确立评估指标与执行评估三个核心步骤，构成标准化实践框架。

1. 构建评估数据集

构建高质量、有代表性的评估数据集是评估基石，应反映系统在真实场景中的情况。

- 数据来源：综合多方数据，包括真实用户查询（从系统日志收集）、业务专家或产品经理设计的关键与边缘用例，以及利用大型语言模型基于知识库生成的问题。
- 数据集结构：由问题、参考上下文和标准答案三部分组成。问题代表查询类型，参

考上下文为评估检索模块提供基准，标准答案作为评估生成答案的参照。

- 数据集规模与划分：通常包含数百至上千个问题，覆盖多种提问类型。按机器学习最佳实践，划分为训练集、开发集和测试集，分别用于调试、验证和最终评估。

2. 确立评估指标

数据集准备就绪后，需确立可量化评估指标，全面覆盖 RAG 流程各环节。表 5.2 是 RAG 评估指标表。

表 5.2 RAG 评估指标表

评估维度	核心指标	定义与说明	常用方法
检索质量	上下文相关性	衡量检索上下文与问题相关程度及是否含关键信息	命中率（标准答案是否在 Top-K 检索结果中）、平均精度（考虑相关文档排序）
	答案忠实性	衡量答案与上下文事实一致性，避免“幻觉”	忠实度（人工或 LLM 评判员判断答案是否基于上下文，计算公式：忠实问题数量/总问题数）
生成质量	答案相关性	衡量答案是否直接、完整、简洁回答问题	使用 LLM 作为评判员，1~5 分制打分
	语义相似度	衡量生成答案与标准答案语义接近程度	计算生成答案与标准答案嵌入向量余弦相似度
端到端质量	综合评分	结合多个指标形成总体分数	使用 RAGAS 等专业评估框架无参考自动计算

注：RAGAS（Retrieval-Augmented Generation Assessment）是一个专为检索增强生成（RAG）系统设计的评估框架，旨在解决 RAG 系统性能量化评估的难题，通过自动化、多维度的指标计算，为开发者提供系统性能的精准诊断与优化方向。

评估方法有人工评估（黄金标准，成本高）和自动评估（使用 LLM 或框架，可大规模进行），实践中常两者结合。

3. 执行评估

执行评估是将指标应用于数据集，通过运行系统、收集数据并分析完成性能衡量的阶段。

- 实验运行：将评估数据集问题批量输入 RAG 系统，记录响应输出，包括检索上下文、生成答案和检索得分等中间结果。
- 指标计算：依据评估指标体系，将系统输出与参考上下文和标准答案比对计算。如计算命中率分析上下文重合度，用 LLM 判断答案忠实度。
- 结果分析与迭代：汇总指标得分，用可视化工具呈现系统强弱项。进行根因分析，若上下文相关性得分低，优化检索模块；忠实性不理想，优化生成模块；答案相关

性低但忠实性高，可能上下文质量不佳或生成能力不足。根据分析制定优化策略并实施，重复评估流程验证效果，形成持续改进闭环。

通过“构建数据集、确立指标和执行评估”三步，可系统化完成 RAG 系统全面“体检”，准确量化性能，为工程迭代提供数据支持和方向指引，是保证系统成功的关键方法论。

5.6 案例实战：基于 RAG 的智能知识管理与问答系统

前面章节已对智能知识管理与问答系统进行了介绍，例如 1.6.2 节综合案例部分概述了智能知识管理与问答平台的整体功能与界面设计，3.3 节案例实战则详细讲解了该平台问答模块的实现。本节将在前述知识的基础上，开展一个基于 RAG（检索增强生成）技术的智能知识管理与问答系统开发实践，重点实现从 PDF 文档中提取知识、存储、检索以及智能问答等功能。

本案例将从功能需求分析入手，逐步构建知识库、检索器、生成器等核心模块，并首先实现一个基于控制台界面的系统版本（以下简称“控制台版本”），完成系统的整合与运行。

源码：

https://gitee.com/wu-solo_admin_admin/llm-course/tree/master/chapter05/rag_system_demo

5.6.1 智能知识管理与问答系统的功能需求

本案例实现一个基于 RAG 技术的智能知识管理与问答系统，主要功能需求如下：

1. PDF 文档管理

PDF 文档管理是系统的基础功能，负责处理用户上传的知识文档。

- (1) 支持用户上传 PDF 文档：用户可以通过输入文件路径的方式上传 PDF 文档，系统支持输入完整路径或相对路径，并对文件路径进行有效性验证，包括检查文件是否存在、格式是否正确等。
- (2) 自动提取 PDF 中的文本内容：使用 PyMuPDF 库高效提取 PDF 文档中的文本内容，支持多页 PDF 文档的处理，能够准确识别和提取中英文混合文本。
- (3) 智能清洗和格式化文本：对提取的文本进行智能清洗，包括修复数字格式问题（如将"2.5"修复为"2.5"）、去除无效字符和噪声、标准化空格和换行符，确保文本质量。
- (4) 按语义边界进行文本分块：按照句子边界对文本进行分块处理，保证每个文本块的语义完整性。支持设置文本块大小和重叠长度，通过重叠机制保留上下文连贯性，提高检索效果。

2. 向量知识库

向量知识库是 RAG 系统的核心组件，负责文本的向量化和存储。

- (1) 将文本块转换为向量表示：使用 **m3e-base** 中文优化嵌入模型，将文本块转换为高维向量表示。该模型针对中文语义进行了优化，能够准确捕捉文本的语义信息，且完全本地运行，无需联网调用。
- (2) 持久化存储到本地向量数据库：使用 **ChromaDB** 轻量级向量数据库，将向量化后的文本块持久化存储到本地磁盘。系统重启后数据仍然可用，无需重新处理 PDF 文档。
- (3) 支持高效的相似度检索：基于余弦相似度算法，实现高效的相似文本检索。支持设置返回结果数量（**top_k**）和相似度阈值，只返回相关性较高的文本块，提高检索精度。
- (4) 提供知识库管理功能（清空、重建）：支持清空当前知识库，方便用户更换新的知识文档。清空操作会删除所有已存储的向量数据，释放存储空间。

3. 智能问答

智能问答是系统的核心功能，实现基于知识库的智能问答。

- (1) 基于知识库检索相关内容：当用户提出问题时，系统首先将问题转换为向量，然后在知识库中检索最相关的文本块。检索过程快速高效，能够在毫秒级别返回结果。
- (2) 调用大语言模型生成准确回答：将检索到的相关文本块作为上下文，构造约束性 **Prompt**，调用 **DeepSeek** 大语言模型生成准确、简洁的回答。**Prompt** 设计遵循“只答所见”原则，确保回答基于知识库内容。
- (3) 无相关内容时自动切换到大语言模型直接回答：当知识库中没有找到相关信息（相似度低于阈值）时，系统自动切换到大语言模型直接回答模式，利用模型自身的知识库回答用户问题，保证用户体验。
- (4) 支持相似度阈值过滤：用户可以设置相似度阈值，只有相似度高于阈值的检索结果才会被使用。这一机制可以有效过滤不相关的内容，提高回答的准确性。

4. 用户界面

用户界面提供友好的交互体验，降低使用门槛。

- (1) 提供简洁的控制台交互界面：采用 **Python** 控制台作为用户界面，无需安装额外的 **Web** 服务器或前端框架。界面设计简洁明了，适合初学者学习和使用。
- (2) 菜单式操作，降低使用门槛：采用菜单式操作方式，用户只需输入数字选择相应功能。每个操作都有清晰的提示和引导，即使没有技术背景的用户也能轻松使用。
- (3) 实时显示处理进度：在 **PDF** 处理、向量化存储等耗时操作中，系统实时显示处理

进度，包括当前步骤、处理状态等信息，让用户了解系统运行状态。

- (4) 清晰的反馈信息：每个操作完成后，系统都会提供清晰的反馈信息，包括操作结果、错误提示、下一步建议等。错误信息详细具体，帮助用户快速定位和解决问题。

5.6.2 智能知识管理与问答平台的总体设计

基于 5.6.1 节所述的功能需求，本节对系统进行总体设计。本系统以 RAG 技术为核心，由知识库、检索器、生成器三大核心模块构成，并辅以文档处理层与交互表现层，共同形成“文档处理→知识库存储→检索器匹配→生成器回答→交互展示”的端到端流程。各模块职责明确、数据流转顺畅，既符合 RAG 技术架构的核心范式，又精准适配 PDF 知识库问答的业务场景。

系统设计遵循“高内聚、低耦合”原则，通过功能关联性分析、数据流向梳理及职责边界划分，合理拆解系统模块，确保架构清晰、易于扩展，同时保障各环节衔接紧密、性能稳定。

1. 模块划分

基于 RAG 技术“知识构建 - 检索匹配 - 答案生成”的核心逻辑，结合业务场景需求，将系统拆解为五大模块。各模块既覆盖 RAG 全流程关键环节，又具备独立职责边界，确保功能复用性与可维护性。具体模块划分如表 5.1 所示。

模块	核心功能	输入/输出	关联文件	划分依据
知识库模块	PDF 上传、文本提取、格式清洗、语义分块、文本标准化	输入：PDF 文件； 输出：标准化文本块列表	pdf_processor pypdf_handler.py	适配 RAG 知识构建环节，处理对象统一（PDF）、为检索器提供高质量输入；不依赖其他业务逻辑，内聚性强
检索器模块	文本向量化、向量存储、相似度检索、结果排序、知识库管理	输入：文本块 / 用户问题； 输出：Top-N 相关文本块	vector_store.py	技术领域统一（向量计算）、性能敏感（直接影响问答响应速度）；可独立替换向量数据库，适配不同规模场景
生成器模块	Prompt 智能构造、LLM 调用、回答优化、上下文管理	输入：用户问题 + 相关文本块；输出：结构化回答	generator.py	技术独立（聚焦 LLM+Prompt 工程）、逻辑复杂（需处理单轮 / 多轮对话）；为系统核心价值载体，回答质量直接决定用户体验

UI 界面模块	菜单展示、输入验证、进度反馈、回答渲染、操作日志	输入：用户操作 / 系统结果； 输出：可视化反馈	console_app.py menu_handler.py qa_handler.py cli_utils.py	关注点独立（仅负责交互，不涉及业务逻辑）；可灵活替换为 Web/GUI 等形态，复用性低且可独立测试
主程序入口	模块初始化、流程调度、异常捕获、资源释放、配置加载	输入：系统启动指令；输出：运行状态 / 异常信息	main.py	作为 RAG 端到端流程的“中枢”，统一协调各模块交互；避免模块间直接依赖，降低耦合度

2. 系统架构

该智能知识检索系统架构分三层：

- 用户界面层：UI 模块提供 Web、移动端、API 接口；
- 业务逻辑层：含知识库、检索器、生成器模块，搭配辅助服务，对接外部数据源与 AI 服务；
- 核心控制层：主程序入口负责系统调度、流程控制等。

具体如图 5.2 所示。

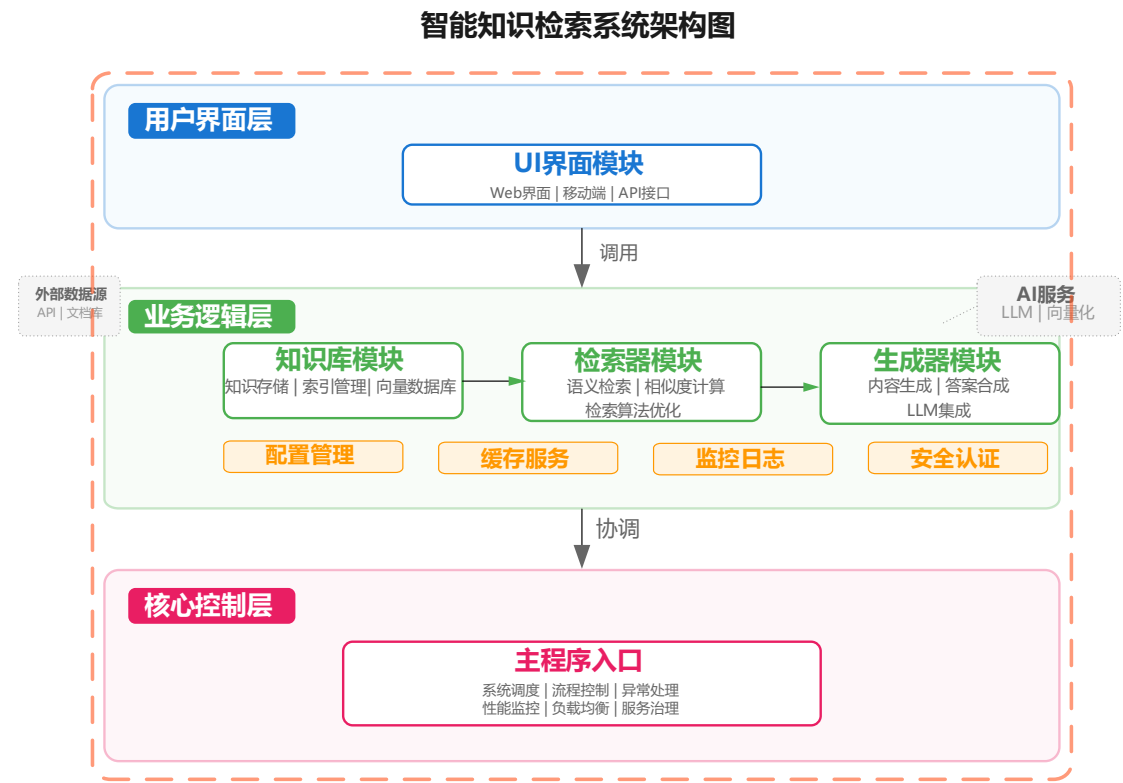


图 5.2 系统架构图

3. 核心数据流

（1）PDF 处理流程

用户输入 PDF 路径 → UI 界面模块（验证输入）→ 知识库模块（文本处理）→ 检索器模块（向量化存储）
→ UI 界面模块（反馈结果）

（2）智能问答流程

用户输入问题 → UI 界面模块（获取输入）→ 检索器模块（问题向量化+相关文本检索）→ 生成器模块
（Prompt+LLM 调用）→ UI 界面模块（展示回答）

5.6.3 知识库模块的实现

知识库模块是整个系统的核心存储层，负责处理 PDF 文档的文本提取、清洗、分块，并将文本块向量化后存储到向量数据库中。本模块通过 `pdf_processor.py` 和 `vector_store.py` 两个文件实现，模块结构如下。

```
knowledge_base/
├── __init__.py      # 模块初始化
├── pdf_processor.py  # PDF 文本处理
└── pdf_handler.py   # PDF 上传处理
```

1. PDF 文本处理（pdf_processor.py）

程序清单 5.11（`pdf_processor.py`）实现了 PDF 文本提取、清洗、分块三个核心功能。

程序清单 5.11: `pdf_processor.py`。

```
1. import re
2. import fitz # PyMuPDF
3.
4. def extract_text_from_pdf(pdf_file):
5.     """提取 PDF 文本"""
6.     with fitz.open(pdf_file) as doc:
7.         return "".join([page.get_text() for page in doc])
8.
9. def clean_text(text):
10.    """清洗文本"""
11.    text = re.sub(r'(\d+)\s*[.,]\s*(\d+)', r'\1.\2', text) # 修复数字格式
12.    text = re.sub(r'\s+', ' ', text).strip() # 标准化空格
13.    return re.sub(r'[\u4e00-\u9fa5a-zA-Z0-9\s,。！？；：“”’（）【】《》、.]', '',
14.                  text)
```

```

14.
15. def split_text_into_chunks(text, chunk_size=500, chunk_overlap=50):
16.     """按句子分块"""
17.     sentences = [s.strip() for s in re.split(r'[。！？；。]', text) if s.strip()]
18.     chunks, current = [], ""
19.     for sent in sentences:
20.         if len(current) + len(sent) > chunk_size and current:
21.             chunks.append(current)
22.             current = current[-chunk_overlap:] if chunk_overlap > 0 else ""
23.             current += sent + "。"
24.     if current:
25.         chunks.append(current)
26.     return chunks

```

主要代码的说明：

1.extract_text_from_pdf: 使用 PyMuPDF 提取 PDF 文本 `fitz.open()` 打开 PDF，`page.get_text()` 提取每页文本。

2.clean_text: 通过正则表达式修复数字格式错误，过滤特殊字符，精简空格和换行，提升文本质量。

3.split_text_into_chunks: 按句子分块，支持重叠；按标点分割句子，控制块大小，通过重叠保留上下文，提升后续检索的准确性。

分块算法说明：

1. 句子分割：按中文标点（。！？；）分割文本。
2. 块构建：逐个添加句子，超过大小时保存当前块。
3. 重叠机制：新块包含上一块的末尾部分，保证语义连贯。

2. PDF 处理模块 (pdf_handler.py)

该文件基于 Chroma 向量数据库实现文本块的向量化存储、相似性检索，代码及关键说明如下：

程序清单 5.12: pdf_handler.py (核心代码)

```

1. import os
2. from knowledge_base.pdf_processor import extract_text_from_pdf, clean_text,
   split_text_into_chunks
3. from ui.cli_utils import print_section_header
4.
5. def upload_pdf(vector_store):
6.     """上传 PDF 到知识库"""
7.     print_section_header("上传 PDF 文档")
8.     file_path = input("文件路径: ").strip().strip('"').strip("'")

```

```

9.         if not file_path or not os.path.exists(file_path) or not
            file_path.lower().endswith('.pdf'):
10.             print("错误: 文件路径无效!")
11.             return None, False
12.
13.         print(f"\n 正在处理: {file_path}")
14.         with open(file_path, 'rb') as f:
15.             text = extract_text_from_pdf(f)
16.
17.         chunks = split_text_into_chunks(clean_text(text))
18.         vector_store.clear_collection()
19.         vector_store.add_text_chunks(chunks)
20.
21.         print(f"\n✓ 成功加载! 共{len(chunks)}个文本块")
22.         return os.path.basename(file_path), True

```

关键功能说明:

- 获取并验证 PDF 文件路径
- 调用 pdf_processor 提取、清洗、分块文本
- 向量化存储到知识库。

5.6.4 检索器模块的实现

检索器模块是连接问题与知识库的桥梁,负责将问题向量化后,从知识库中检索出最相关的文本块。本案例中,检索器模块的功能已集成在 `vector_store.py` 的 `ChromaVectorStore` 类中,模块结构如下。

```

retriever/
├── __init__.py      # 模块初始化
└── vector_store.py  # 向量数据库

```

1. 检索流程

问题向量化: 调用 `get_embedding` 函数,将问题转换为与知识库中文本块同维度的向量。

相似性计算: 使用 `Chroma` 的 `query` 方法,基于余弦相似度计算问题向量与知识库中所有文本块向量的相似度。

结果返回: 返回相似度最高的 `top_k` 个文本块及其相似度值,作为后续生成答案的上下文。

2. 检索器的调用示例

程序清单 5.13: `vector_store.py`

```

1. import chromadb
2. from chromadb.config import Settings
3. from sentence_transformers import SentenceTransformer
4.
5. embedding_model = SentenceTransformer("moka-ai/m3e-base")
6.
7. class ChromaVectorStore:
8.     def __init__(self):
9.         self.client = chromadb.PersistentClient(path="./chroma_db")
10.        self.collection = self.client.get_or_create_collection("pdf_knowledge_base")
11.
12.    def add_text_chunks(self, chunks):
13.        """存储文本块"""
14.        embeddings = [embedding_model.encode(c).tolist() for c in chunks]
15.        ids = [f"chunk_{i}" for i in range(len(chunks))]
16.        self.collection.add(ids=ids, embeddings=embeddings, documents=chunks)
17.
18.    def similarity_search(self, query, top_k=3, threshold=0.7):
19.        """相似度检索"""
20.        query_emb = embedding_model.encode(query).tolist()
21.        results = self.collection.query(query_embeddings=[query_emb],
22.                                       n_results=top_k)
23.        return [(doc, round(1-dist, 4)) for doc, dist in zip(results["documents"][0],
24.                                                           results["distances"][0])
25.               if 1-dist >= threshold]

```

代码说明：

- 使用 m3e-base 模型生成向量。
- ChromaDB 持久化存储。
- 余弦相似度检索，支持阈值过滤。

5.6.5 生成器模块的实现

生成器模块负责基于检索到的上下文文本块，调用大语言模型（LLM）生成精准的回答。本案例中，生成器模块通过程序清单 5.14 generator.py 实现的，使用 DeepSeek API 作为大语言模型服务，模块结构如下。

```

generator/
├── __init__.py      # 模块初始化
└── generator.py     # LLM 生成器

```

程序清单 5.14: generator.py

```

1. import requests

```

```

2. import os
3.
4. API_KEY = os.getenv("DEEPSEEK_API_KEY")
5. BASE_URL = os.getenv("DEEPSEEK_BASE_URL")
6.
7. def call_llm(prompt):
8.     """调用 LLM API"""
9.     resp = requests.post(f"{BASE_URL}/chat/completions",
10.                          headers={"Authorization": f"Bearer {API_KEY}"},
11.                          json={"model": "deepseek-chat", "messages": [{"role": "user",
12.                               "content": prompt}],
13.                               "temperature": 0.3})
14.     return resp.json()["choices"][0]["message"]["content"]
15.
16. def generate_answer(query, context_chunks):
17.     """生成回答"""
18.     if not context_chunks:
19.         return call_llm(f"回答问题: {query}")
20.     context = "\n".join([f"参考{i+1}: {c[0]}" for i, c in enumerate(context_chunks)])
21.     return call_llm(f"根据参考资料回答: \n{context}\n\n 问题: {query}\n 回答: ")

```

代码说明:

- 调用 DeepSeek API 生成回答
- 有上下文时基于知识库回答
- 无上下文时直接回答。

5.6.6 UI 界面

UI 界面模块负责用户交互，提供简洁的控制台操作界面。模块结构如下。

ui/

```

├── __init__.py      # 模块初始化
├── console_app.py   # 控制台应用
├── menu_handler.py  # 菜单处理
├── qa_handler.py    # 问答处理
└── cli_utils.py     # 控制台工具

```

1. 控制台工具 (cli_utils.py)

程序清单 5.15: cli_utils.py

```

1. import os
2.

```



```
3. def clear_screen():
4.     os.system('cls' if os.name == 'nt' else 'clear')
5.
6. def print_menu_header(title):
7.     clear_screen()
8.     print("=" * 60)
9.     print(f"{title:^60}")
10.    print("=" * 60)
11.    print()
12.
13. def print_section_header(title):
14.     print(f"\n{'='*60}\n{title}\n{'='*60}\n")
15.
16. def print_divider():
17.     print("-" * 60)
```

代码说明：

- 控制台格式化输出工具
- 清除屏幕、打印标题、分隔线

2. 菜单处理 (menu_handler.py)

程序清单 5.16: menu_handler.py

```
1. from ui.cli_utils import clear_screen, print_menu_header
2.
3. def show_menu(current_pdf, chunks_loaded):
4.     """显示菜单"""
5.     clear_screen()
6.     print_menu_header("RAG 问答系统")
7.     status = f"当前: {current_pdf}" if current_pdf else "当前: 未加载"
8.     print(f"{status}\n")
9.     print("1. 上传 PDF  2. 提问  3. 清空  4. 退出")
10.    print("=" * 60)
11.
12. def get_user_choice():
13.     return input("\n 请选择 (1-4) : ").strip()
```

代码说明：

- 显示主菜单和当前状态
- 获取用户选择

3. 问答处理 (qa_handler.py)

程序清单 5.17: qa_handler.py

```
1. from generator.generator import generate_answer
2. from ui.cli_utils import print_section_header, print_divider
3.
4. def ask_question(vector_store, chunks_loaded):
5.     """处理问答"""
6.     if not chunks_loaded:
7.         print("请先上传 PDF! ")
8.         return
9.
10.    print_section_header("提问")
11.    query = input("问题: ").strip()
12.    if not query:
13.        return
14.
15.    print("\n 检索中...")
16.    chunks = vector_store.similarity_search(query)
17.
18.    print("生成回答...")
19.    answer = generate_answer(query, chunks)
20.
21.    print_divider()
22.    print(f"回答: {answer}")
23.    print_divider()
24.    input("\n 按回车继续...")
```

代码说明:

- 获取用户问题
- 检索知识库
- 生成并显示回答

4. 控制台应用 (console_app.py)

程序清单 5.18: console_app.py

```
1. from retriever.vector_store import ChromaVectorStore
2. from ui.menu_handler import show_menu, get_user_choice
3. from knowledge_base.pdf_handler import upload_pdf
4. from ui.qa_handler import ask_question
5.
6. class RAGConsoleApp:
7.     def __init__(self):
```

```

8.         self.vector_store = ChromaVectorStore()
9.         self.current_pdf = None
10.        self.chunks_loaded = False
11.
12.    def run(self):
13.        while True:
14.            show_menu(self.current_pdf, self.chunks_loaded)
15.            choice = get_user_choice()
16.
17.            if choice == '1':
18.                self.current_pdf, self.chunks_loaded = upload_pdf(self.vector_store)
19.            elif choice == '2':
20.                ask_question(self.vector_store, self.chunks_loaded)
21.            elif choice == '3':
22.                self.vector_store.clear_collection()
23.                self.current_pdf, self.chunks_loaded = None, False
24.            elif choice == '4':
25.                print("再见！")
26.                break

```

代码说明：

- 主应用类，协调各模块
- 处理用户菜单选择
- 维护应用状态

5.6.7 程序主入口和运行

主程序入口（main.py）

程序清单 5.19: main.py

```

1.  from ui.console_app import RAGConsoleApp
2.
3.  if __name__ == '__main__':
4.      print("=" * 60)
5.      print("          RAG 问答系统 - 控制台版本")
6.      print("=" * 60)
7.      print("\n 初始化中...")
8.
9.      try:
10.         app = RAGConsoleApp()
11.         print("初始化完成！")
12.         app.run()

```

```
13.         except Exception as e:
14.             print(f"错误: {e}")
```

代码说明:

- 系统入口
- 初始化应用
- 启动主循环

5.6.8 运行示例

在 VS code 或 Trae 终端运行 `python main.py` 程序, 会出现如下运行界面, 根据提示进行操作。

按照上面的提示回车, 出现下面的菜单选择。

```
(llm-course) PS E:\llm-course\rag_system_dbllm> python main.py
```

```
=====
                        RAG 问答系统 - 控制台版本
=====

正在初始化...
- 加载嵌入模型 (首次运行会自动下载)
- 初始化向量数据库

请稍候...
✓ 初始化完成!
按回车键开始使用...
```

说明: 首次运行时, 系统会自动下载 `m3e-base` 中文嵌入模型 (约 1.2GB), 这是正常现象, 后续运行将直接加载本地模型。

```
=====
请输入选项 (1-4): 1
=====

上传 PDF 文档
=====

请输入 PDF 文件的完整路径 (例如: D:\docs\knowledge.pdf)
或者直接输入文件名 (如果文件在当前目录下)
文件路径: 第 1 章 自然语言处理概述.pdf
正在处理文件: 第 1 章 自然语言处理概述.pdf
1. 提取文本...
2. 清洗文本...
3. 分块处理...
   共生成 72 个文本块
4. 向量化并存储到知识库...
```

成功存储 72 个文本块

✓ 知识库已成功加载！

文件名：第 1 章 自然语言处理概述.pdf

文本块数量：72

按回车键继续...

=====
请输入选项（1-4）： 1
=====

上传 PDF 文档
=====

请输入 PDF 文件的完整路径（例如：D:\docs\knowledge.pdf）

或者直接输入文件名（如果文件在当前目录下）

文件路径：第 1 章 自然语言处理概述.pdf

正在处理文件：第 1 章 自然语言处理概述.pdf

1. 提取文本...

2. 清洗文本...

3. 分块处理...

共生成 72 个文本块

4. 向量化并存储到知识库...

成功存储 72 个文本块

✓ 知识库已成功加载！

文件名：第 1 章 自然语言处理概述.pdf

文本块数量：72

按回车键继续...

在上面的运行界面，选择菜单 1，输入上传 pdf 文档的文件路径，系统开始对上传的 pdf 文件进行提前文本、清洗文本、对文件进行分块，然后存储到向量数据库中。接着根据提示回车。

当前知识库：第 1 章 自然语言处理概述.pdf

已加载文本块：是

请选择操作：

1. 上传 PDF 文档（输入文件路径）

2. 提问

3. 清空知识库

4. 退出

```
=====
请输入选项（1-4）： 2
=====
```

```
提问
=====
```

```
请输入您的问题（输入 'quit' 返回主菜单）：
```

```
问题：什么是 NLP?
```

```
正在检索相关知识...
```

```
找到 3 条相关资料
```

```
正在生成回答...
```

```
-----
回答：
```

```
-----
根据参考资料，NLP 是自然语言处理（参考资料 1）。它是计算机科学与语言学交叉的一个重要领域，其研究任务涵盖
多个方面（参考资料 2）。
-----
```

选择菜单 2，输入提问的问题“什么是 NLP?”，接着系统到向量数据库进行检索，在知识库中检索相关文本，然后调用大语言模型生成回答，最后显示回答结果生成答案输出到界面。

本章小结

本章主要讲解了向量数据库与高级 RAG 技术的相关知识及应用。

对于向量数据库而言，阐述了其作为 RAG 系统记忆基石的核心作用，详细介绍了其从传统数据库演变而来的原理、主流生态，并通过实战演示了 ChromaDB 和 Milvus Lite 的部署与核心数据操作。

对于 RAG 技术而言，深入探讨了作为语义理解基础的嵌入模型，讲解了多种检索策略与结果优化方法，并介绍了重排序、查询改写等高级技巧。最后，通过一个完整的案例，将所有知识点串联起来，展示了如何从零开始构建一个企业级知识库问答系统。

阅读文献和资料

[1] 罗云. 从零构建向量数据库[M]. 北京：人民邮电出版社，2024.

[2] 梁楠. 向量数据库：大模型驱动的智能检索与应用[M]. 北京：清华大学出版社，2025.

[3] 龙中华 . LangChain 实战派：大语言模型+LangChain+向量数据库[M]. 北京：电子工业出版社，2024.

[4] 开源大语言模型社区与文档. <https://huggingface.co/>

<https://docs.trychroma.com/docs/overview/introduction>.

[5] 从零开始的向量数据库原理与实践教程：

<https://github.com/datawhalechina/easy-vectordb>.

[5] RAG 五大项目（LLM+企业级数据库+embedding+问答系统+Agent）教程，从入门到实战：

[6] https://www.bilibili.com/video/BV1vGycBuEZq?p=22&vd_source=dcbf7406e8d9a3afab6d66c0ed29675d

[7] Patterson, J., et al. AI and Machine Learning for Coders: A Programmer's Guide to Artificial Intelligence[M]. O'Reilly Media, 2021.

[8] ChromaDB 官方文档. <https://docs.trychroma.com>.

[9] **All-in-RAG | 大模型应用开发实战一：RAG 技术全栈指南：**
<https://datawhalechina.github.io/all-in-rag/#/>

[10] RAG 学习指南：<https://llm-stacks.com/docs/intro>

[11] RAG 的三大范式和技术演进：

https://www.toutiao.com/video/7612948430670922240/?app=news_article&category_new=__all__&module_name=Android_tt_others&share_did=MS4wLjACAAACgRJDOztD9LdU9fr8W2cFqD7YaTx5qegccwLwkSSW6Yy6iOKXQQ8SWEzxyse8ZkB&share_uid=MS4wLjABAAAANRWtloFje04QBsk9zhZIAfAwC-G1Vr4Mu65XhJVpyezv-rQPvfdsaV1fpiT7u2rd×tamp=1773471238&tt_from=wechat&upstream_biz=Android_wechat&utm_campaign=client_share&utm_medium=toutiao_android&utm_source=wechat&share_token=450b7642-364b-498d-9df9-380de6a2cb56&source=m_redirect&wid=1773667040177